Predicting Credit Card Payment Default Using Client Demographics and Payment History

AAI-500: Probability and Statistics for Artificial Intelligence

University of San Diego, Shiley-Marcos School of Engineering

Final Team Project, Group 02

Abhineet Sood

Ravi Kanth Vuddagiri

October 4, 2026

GitHub repository: https://github.com/asood-usd/AAI500-credit-default

1. Introduction

When a credit card client stops paying, the bank loses the unpaid balance. If the bank could spot risky clients a month early, it could do something before the loss happens, like send a reminder, offer a payment plan, or hold off on a limit increase. The question is which clients to focus on.

For this project we used the **Default of Credit Card Clients** dataset from the UCI Machine Learning Repository (Yeh & Lien, 2009). It has **30,000 clients of a bank in Taiwan** from April to September 2005. For each client we know the credit limit, four demographic variables (sex, education, marital status, age), and six months of history: the repayment status, the bill amount, and the amount paid each month. The target is whether the client **defaulted on the October 2005 payment**. About 22% did.

We wanted to answer three questions:

1. **Which factors are related to default, and how strongly?** We used chi-squared tests, t tests, a two-proportion z test and confidence intervals, like in Modules 4 and 5.
2. **How well can we predict default?** Our main model is a logistic regression, because its coefficients can be explained as odds ratios. We compared it with a decision tree, a random forest and gradient boosting.
3. **How should a bank use the predictions?** We chose a cutoff based on what each kind of mistake costs, instead of using the usual 0.5.

Main result. How a client has been paying matters much more than who the client is. A client who was two or more months late in September has about **6 times the odds** of defaulting compared with a client who paid in full, after accounting for everything else. Our logistic regression reaches a test ROC-AUC of **0.764**, only about 0.01 below the best model (a random forest, 0.773). With a cost-based cutoff it catches **71% of defaulters** instead of 34%, and lowers the expected cost by about **25%**.

2. Data Cleaning and Preparation

2.1 Data quality checks

The raw file has 30,000 rows and 25 columns (an ID, 23 predictors and the target). We checked for missing values, duplicates, and category codes that don't match the dataset documentation.

- **Missing values:** none.
- **Duplicates:** 35 rows are exact copies of another row once the ID column is removed.

- **Undocumented codes:** the documentation lists EDUCATION as 1–4 and MARRIAGE as 1–3, but the data also has EDUCATION = 0, 5, 6 (345 rows) and MARRIAGE = 0 (54 rows).
- **Repayment status codes:** the documentation describes -1 (paid on time) and 1–9 (months late), but most values are actually -2 and 0. Other studies of this data read -2 as "no balance / card not used" and 0 as "revolving credit" (paid the minimum, not late). These are real account states, not errors.

2.2 Cleaning decisions

Table 1. Cleaning steps (in `src/cleaning.py`).

Issue	Rows	Action	Why
`ID` column	all	Dropped	It's just a row number
Exact duplicates (ignoring ID)	35	Dropped	Would count the same client twice
`EDUCATION` = 0, 5, 6	345	Changed to 4 ("Other")	Undocumented and rare (1.2%)
`MARRIAGE` = 0	54	Changed to 3 ("Other")	Undocumented and rare (0.2%)
`PAY_*` = -2, 0	many	Kept	Real states (no balance, revolving)
`PAY_0` column name	n/a	Renamed `PAY_1`	Lines up with `BILL_AMT1` and `PAY_AMT1`
Target name	n/a	Renamed `DEFAULT`	Shorter

We **kept the outliers**. Very large credit limits (up to NTD 1,000,000) and payments are believable for wealthy clients. Also, 1,930 clients have at least one negative bill, which means they overpaid and had a credit balance. Removing them would throw away real clients.

The clean data has **29,965 rows and 24 columns**, with a **22.13% default rate** (6,630 defaults).

2.3 Feature engineering

For the logistic regression we built 15 features (`src/features.py`). Each choice comes from something we found in the exploratory analysis (Section 3).

Table 2. Features used in the logistic regression.

Feature	How we built it	Why
Credit limit	`LIMIT_BAL` / 10,000	Odds ratio per NTD 10,000 is easier to read
Age	`AGE` / 10	Odds ratio per 10 years
Sex, education, marital status	0/1 dummies	Reference groups: male, graduate school, married
September status	4 groups: paid / no balance (-2, -1), revolving (0), 1 month late, 2+ months late	Strongest predictor; codes 3–8 are rare
Months late, Apr–Aug	Count of `PAY_2`…`PAY_6` ≥ 1	Replaces 5 correlated columns
Utilization	`BILL_AMT1` / `LIMIT_BAL`, kept between -1 and 3	Replaces 6 bill columns with VIF 10–16
Recent payments	log(1 + `PAY_AMT1`), log(1 + `PAY_AMT2`)	Payments are very skewed right

The tree models used the 23 cleaned columns directly, since trees don't need dummies or logs.

2.4 Train/test split

We split the data **70% / 30%** (20,975 training and 8,990 test clients), the same split suggested in Module 5. The split is **stratified**, so both parts keep the 22% default rate, and it uses a fixed random seed (500). We made every modeling choice, including the cutoff, using **5-fold cross-validation on the training set only**. We used the test set once, at the end.

3. Exploratory Data Analysis

3.1 How many clients defaulted?

Only **22.13%** of clients defaulted. A model that always predicts "no default" would already be about 78% accurate, so accuracy by itself is misleading here. We looked at ROC-AUC, precision and recall instead.

3.2 Repayment history

September repayment status is the strongest single pattern in the data (Figure 1). Clients who paid in full or carried a normal revolving balance defaulted 13–16% of the time. One month late raises that to **34%**, and two or more months late to **70%**. The chi-squared test of independence gives $\chi^2 = 5{,}333.29$ with df = 3 and $P \approx 0$.

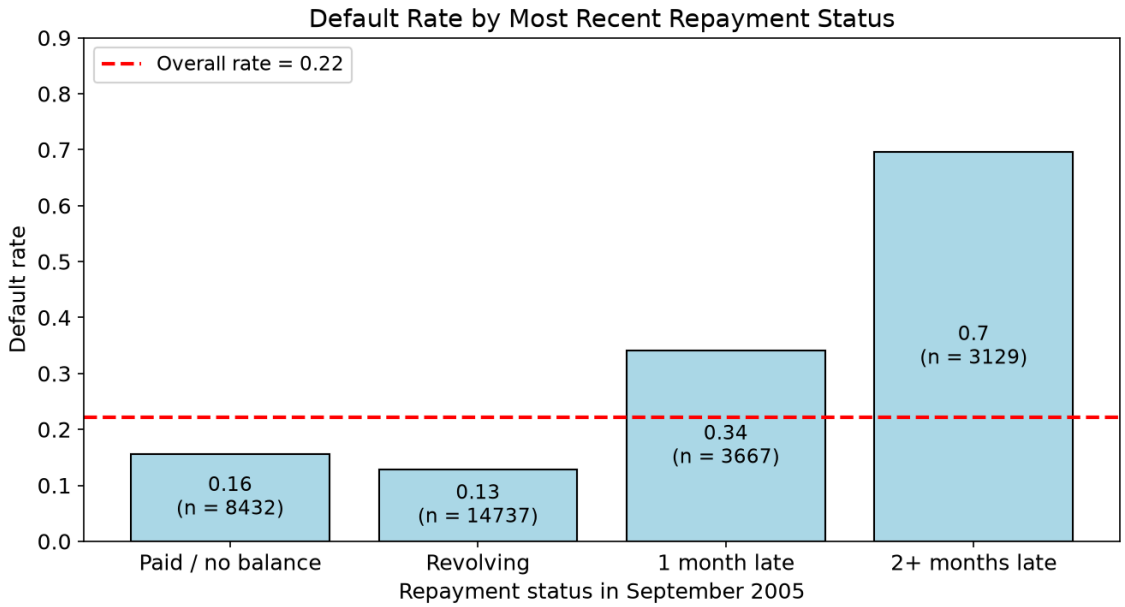


Figure 1. Default rate by September repayment status. The dashed line is the overall rate.

Looking at all six months shows the same thing (Figure 2). Clients who were never late defaulted **12%** of the time, and clients who were late in all six months defaulted **70%** of the time.

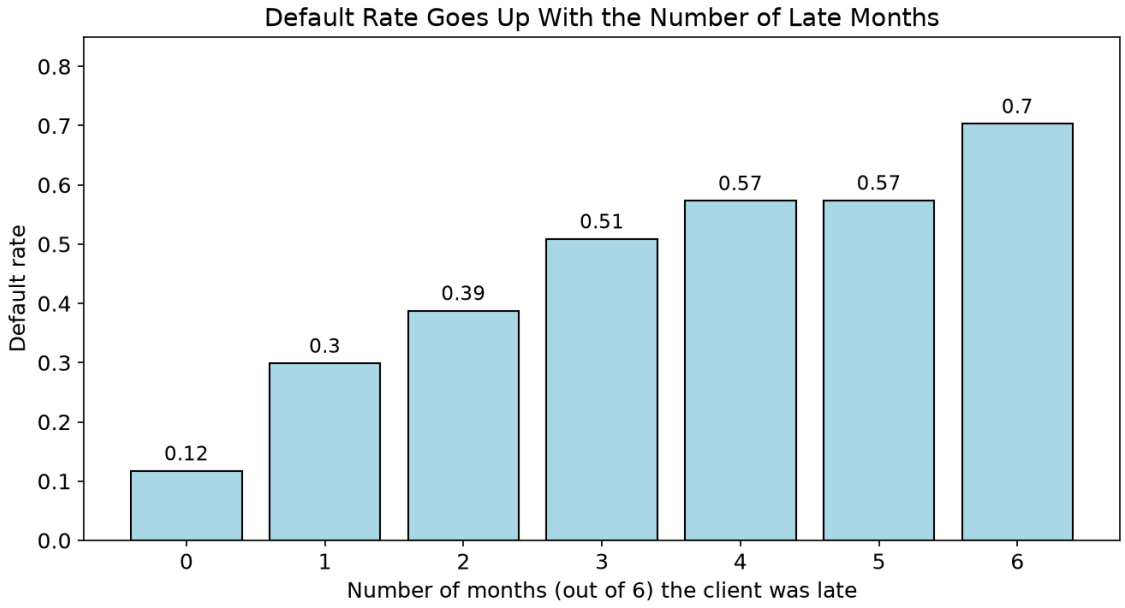


Figure 2. Default rate by the number of months (out of six) a client was at least one month late.

3.3 Credit limit, payments and bills

For each numeric variable we ran a two-sample t test with `equal_var=False` and found the 95% CI for the difference in means (Table 3). The dollar amounts are very skewed, so, like the house prices in Module 4, we also ran the same test on log(1 + amount). It gave the same conclusions every time.

Table 3. Difference in means (default − no default), with 95% CIs.

Variable	Mean, no default	Mean, default	Difference (95% CI)	P-value
Credit limit (NTD)	178,085	129,985	-48,100 (-51,349 to -44,851)	< 0.001
September payment (NTD)	6,315	3,400	-2,915 (-3,241 to -2,589)	< 0.001
August payment (NTD)	6,649	3,392	-3,257 (-3,687 to -2,826)	< 0.001
September bill (NTD)	52,059	48,553	-3,506 (-5,518 to -1,494)	< 0.001
Age (years)	35.42	35.72	0.30 (0.04 to 0.56)	0.02

Defaulters had **lower credit limits** and **paid about half as much** (Figure 3). In September, **28% of defaulters paid nothing at all**, compared with 14% of non-defaulters. The bill difference is significant, but it's only about 7% of the average bill. **Age** is significant at $\alpha = 0.05$, but the difference is only 0.30 years (about 3 to 4 months), which is too small to matter in practice.

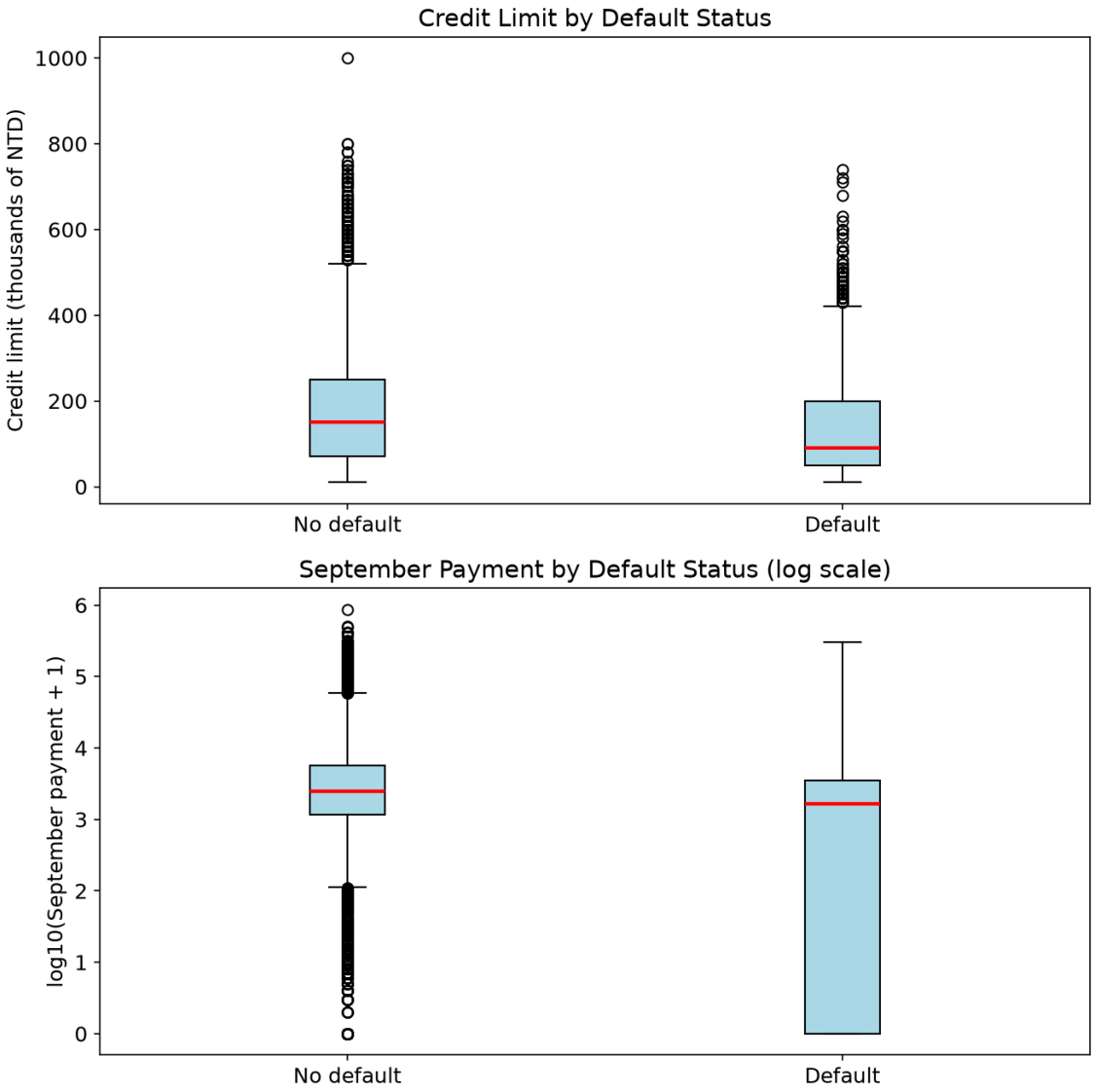


Figure 3. Credit limit (top) and September payment on a log scale (bottom), by default status. Red lines are medians.

3.4 Demographics

Sex, education, marital status and age group are all related to default (all chi-squared P-values below 0.001), but mostly because the sample is so large. The default rates only differ by about 0.03 to 0.06 between groups (education looks bigger only because of the small "Other" group). For example, men defaulted 24.16% of the time and women 20.79%. A two-proportion z test gives $z = 6.88$ ($P < 0.001$), and the 95% CI for the difference is **2.40 to 4.34 percentage points**. The gap is real, but small next to the 20 to 50 point jumps that come with repayment status.

3.5 Correlation and multicollinearity

The six monthly bill amounts are very highly correlated with each other (r = 0.80 to 0.95), and their **variance inflation factors (VIF) are 10 to 16**, above the usual cutoff of 5 to 10. The six repayment status columns are also correlated (r = 0.48 to 0.82). If we put all of these into a logistic regression, we couldn't trust the coefficients. That's why we summarized them (Table 2). After that, **every VIF is below 2.2**.

4. Model Selection

4.1 Logistic regression: does payment behavior add information?

Logistic regression models the log-odds of default as a linear function of the features. `statsmodels` fits it by **maximum likelihood**, the same idea as the likelihood function in Module 4, just with 16 parameters. Each coefficient β turns into an **odds ratio** e^β: how much the odds of default get multiplied when that feature goes up by 1 and the others stay the same.

We fit two models on the training set:

- **Model A (profile only):** credit limit, age, sex, education, marital status (8 features).
- **Model B (profile + behavior):** Model A plus September status, the count of earlier late months, utilization and the two payment amounts (15 features).

We compared them with a **likelihood ratio test**. H_0 says the seven behavior coefficients are all 0. The test statistic is LR = 2(ℓ_B − ℓ_A) = **3,352.19 with df = 7, P ≈ 0**, so we reject H_0. Model B also has a much lower AIC (**18,133.15 vs. 21,471.34**), and its pseudo R^2 goes up from **0.03 to 0.18**. The profile by itself explains very little; payment behavior is what makes the model work. We used Model B from here on.

4.2 Comparison models

To see what we give up by using a simple model we can explain, we compared four other models:

- **Logistic regression on all 23 raw columns**, to check whether our feature work helped.
- **Decision tree** (max depth 5, at least 50 clients per leaf).
- **Random forest** (300 trees). We tried leaf sizes of 5, 10, 25, 50 and 100 with cross-validation. Sizes 25 to 100 all scored about the same, and 50 was the highest.
- **Gradient boosting** (scikit-learn's `HistGradientBoostingClassifier`, the same idea as XGBoost).

We scored every model with **5-fold stratified cross-validation ROC-AUC** on the training set. ROC-AUC is the chance that the model gives a random defaulter a higher score than a random non-defaulter. It doesn't depend on a cutoff, so it's a fair way to compare models when only 22% of clients default.

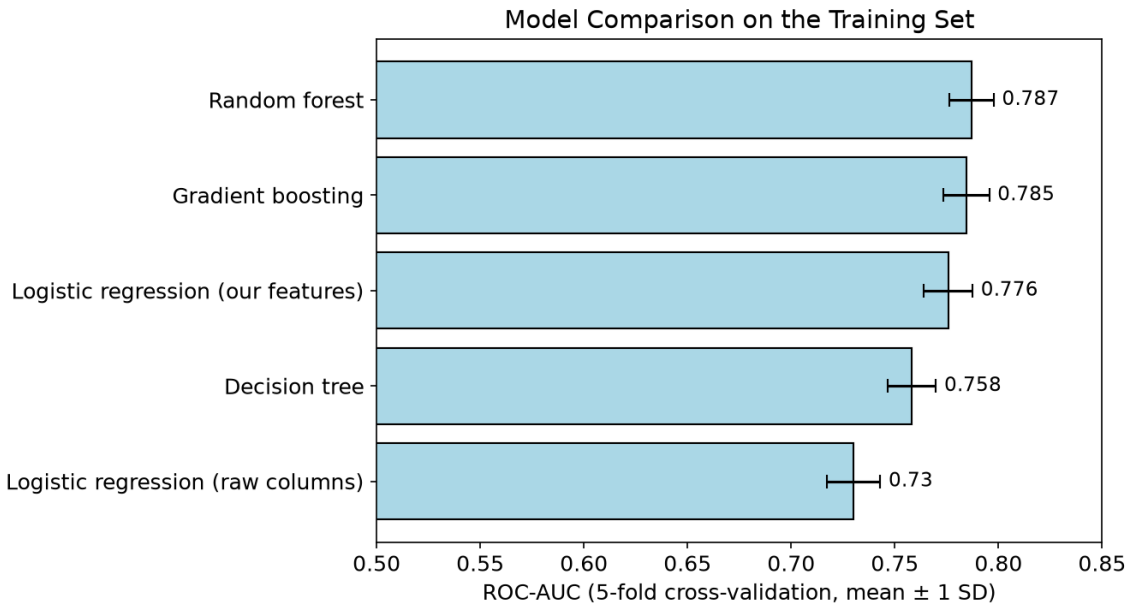


Figure 4. Cross-validated ROC-AUC on the training set (mean ± 1 SD across the 5 folds).

The random forest (0.787) and gradient boosting (0.785) scored highest. Our logistic regression (0.776) was about **0.011 lower**, roughly one fold-to-fold SD. The same logistic regression on the raw columns scored only **0.730**. So **our feature work helped more (+0.046) than switching to a more complex model (+0.011)**.

We took two finalists into the evaluation: the **logistic regression** (the model we can explain) and the **random forest** (the best score).

5. Model Analysis

5.1 What drives default: odds ratios

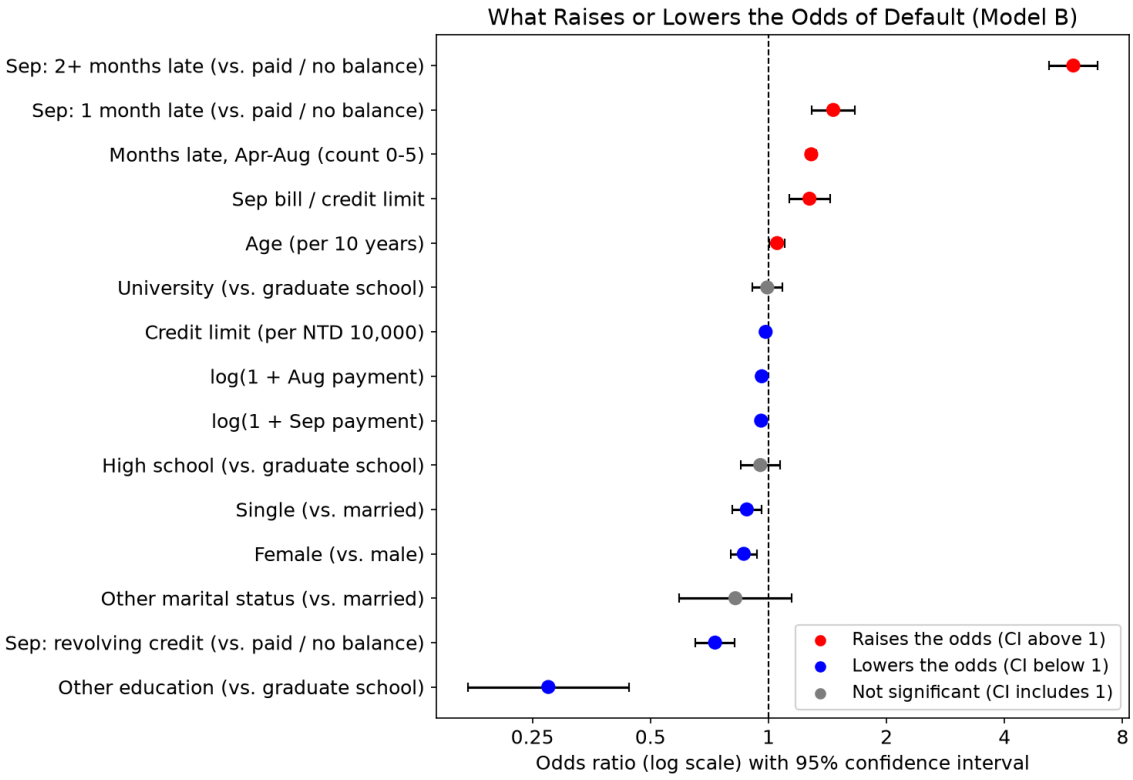


Figure 5. Odds ratios from Model B with 95% confidence intervals (log scale). Red raises the odds, blue lowers them, gray is not significant.

Table 4. Selected odds ratios from Model B (the full table is in `reports/tables/odds_ratios.csv`).

Feature	Odds ratio	95% CI	P-value
Sep: 2+ months late (vs. paid / no balance)	**5.99**	5.19 to 6.91	< 0.001
Sep: 1 month late	1.46	1.29 to 1.66	< 0.001
Each earlier late month (Apr–Aug)	1.29	1.25 to 1.33	< 0.001
Utilization (bill / limit)	1.27	1.13 to 1.43	< 0.001
Credit limit, per NTD 10,000	0.983	0.980 to 0.987	< 0.001
log(1 + Sep payment)	0.958	0.946 to 0.970	< 0.001
Sep: revolving credit	0.73	0.65 to 0.82	< 0.001
Female (vs. male)	0.87	0.80 to 0.93	< 0.001
Single (vs. married)	0.88	0.81 to 0.96	0.004
Age, per 10 years	1.05	1.00 to 1.10	0.032
University (vs. graduate school)	0.99	0.91 to 1.08	0.872

What the main results mean:

- **Recent lateness matters most.** Being 2+ months late in September multiplies the odds by about 6. A client who was late in all five earlier months has about $1.29^5 \approx$ **3.5 times** the odds of one who was never late.
- **Credit limit:** NTD 100,000 more limit gives $0.983^{10} \approx$ **0.84**, about 16% lower odds. Part of this is probably because the bank already gives bigger limits to clients it trusts.
- **Payments:** paying 10 times more in September lowers the odds by about 9%.
- **Revolving vs. paid in full:** revolving clients have lower odds than the "paid / no balance" group. This surprised us. We think it's because code -2 includes inactive cards, which may behave differently.
- **Demographics** have small effects. Education (other than the small "Other" group, which includes the undocumented codes we merged) is not significant once payment behavior is in the model.

Do we need demographics at all? We refit the model without sex, marital status and age. The test AUC went from **0.7637 to 0.7636**. These variables add basically nothing to the predictions, so a bank could leave them out. That matters for fairness and legal reasons.

The random forest agrees. Its six most important columns are the six monthly repayment statuses, with September (PAY_1) far ahead of the rest, followed by the September payment, credit limit and bill. No demographic variable makes its top 10. Two very different models agreeing on what matters makes us more confident in this finding.

5.2 Test-set performance

Table 5. Test-set results (8,990 clients) at the usual 0.5 cutoff.

Model	ROC-AUC	Accuracy	Precision	Recall	F1
Gradient boosting	0.773	0.816	0.658	0.355	0.461
Random forest	0.773	0.814	0.652	0.346	0.452
Logistic regression (our features)	**0.764**	0.814	0.654	0.338	0.445
Decision tree	0.750	0.815	0.664	0.332	0.443
Logistic regression (raw columns)	0.708	0.807	0.679	0.242	0.357

The test AUCs are close to the cross-validation scores, so the models weren't overfit (Figure 6). At the 0.5 cutoff, though, every model **only flags about a third of the defaulters**. Section 5.4 fixes this.

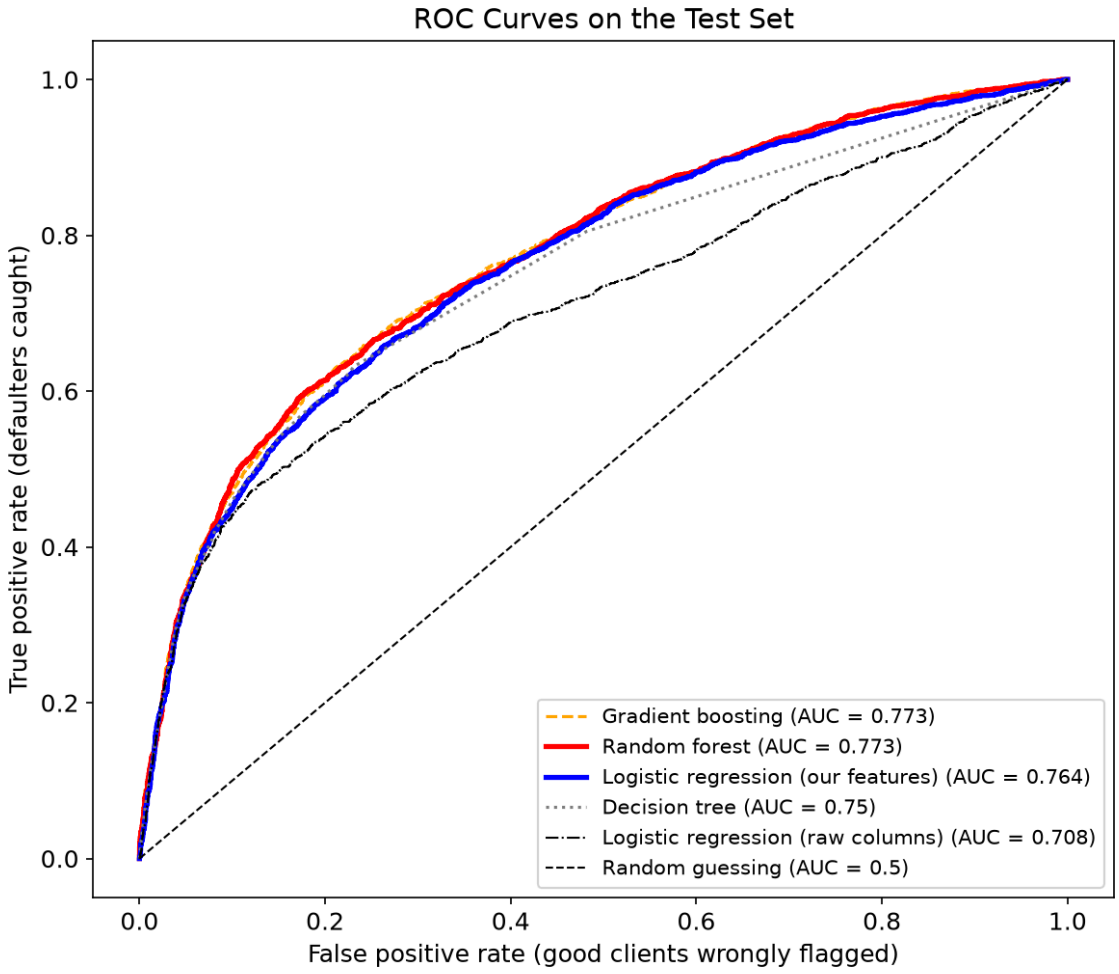


Figure 6. ROC curves on the test set.

5.3 Is the random forest really better?

We bootstrapped the test set 2,000 times and recomputed both AUCs on the same resampled clients each time. The 95% CI for the random forest's advantage is **0.003 to 0.015**, and the random forest is higher in 99.8% of the resamples. So the difference is real, but at about **0.01 AUC it's small in practice**.

5.4 Choosing a cutoff

A 0.5 cutoff treats both kinds of mistakes as equally bad. For a bank, missing a defaulter (losing the balance; the median September bill among defaulters was about NTD 20,000) is much worse than a false alarm (a reminder call or a temporary hold for a good client). We **assumed a missed default costs 5 times as much as a false alarm**, and picked the cutoff with the lowest expected cost on the **out-of-fold training predictions**.

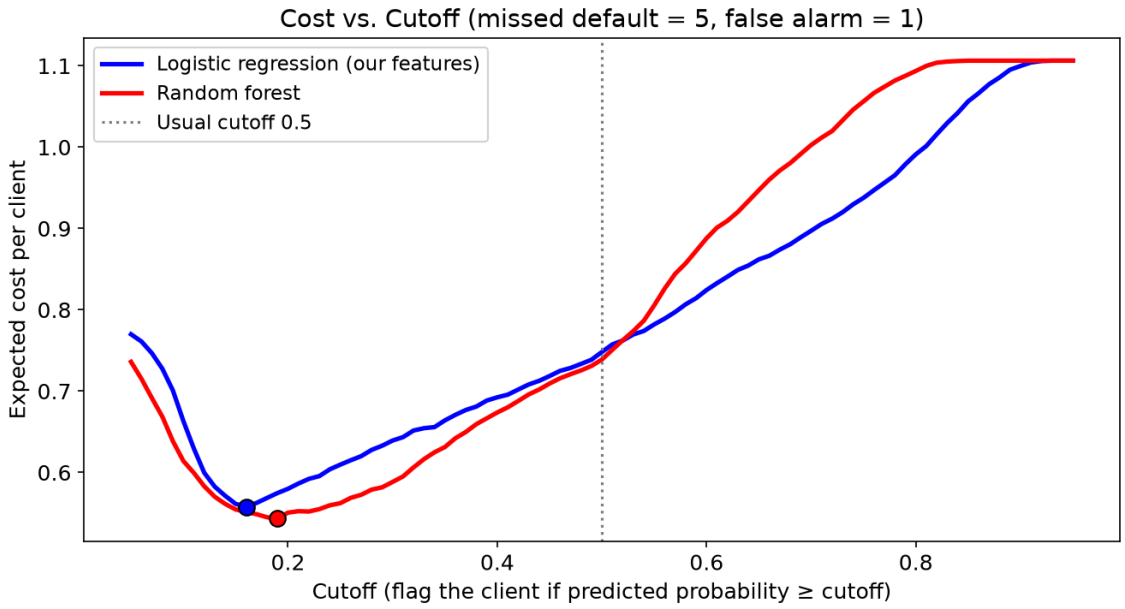


Figure 7. Expected cost per client vs. cutoff (training set, out-of-fold predictions).

The best cutoff was **0.16** for the logistic regression and **0.19** for the random forest. This matches theory: if the probabilities are accurate, flagging a client is worth it when $p \times 5 > (1 - p) \times 1$, which works out to $p > 1/6 \approx 0.17$ (Elkan, 2001).

Table 6. Logistic regression on the test set at 0.5 vs. the cost-based cutoff.

Cutoff	Precision	Recall	Clients flagged	Expected cost per client
0.50	0.65	0.34	11%	0.772
0.16	0.38	**0.71**	41%	**0.575**

Lowering the cutoff **doubles the share of defaulters caught (34% → 71%)** and lowers the expected cost by about **25%**. The random forest at its own cutoff costs 0.567 per client, about 1.4% less than the logistic regression.

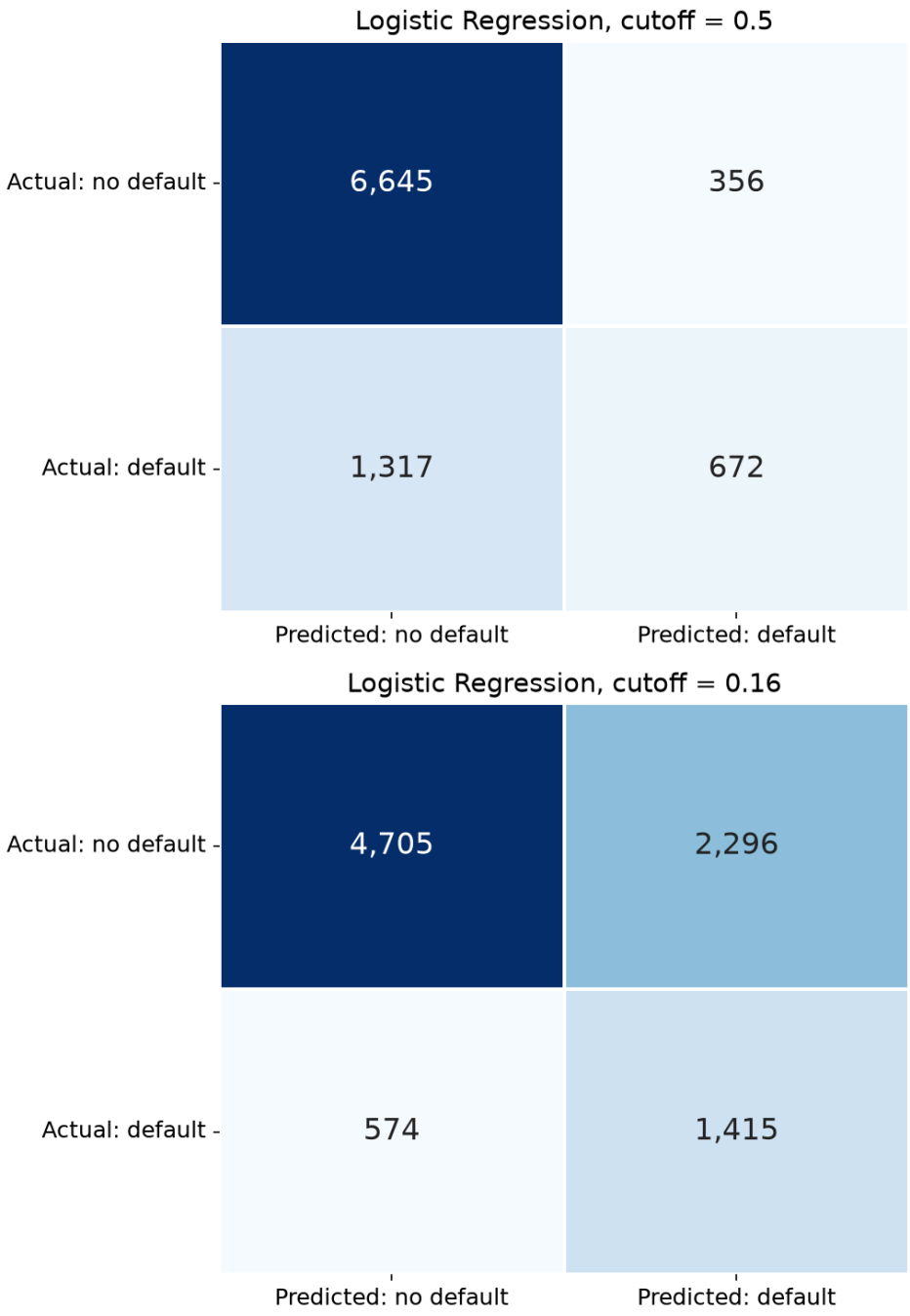


Figure 8. Logistic regression confusion matrices on the test set at cutoffs 0.5 and 0.16.

How much does the cost assumption matter? We don't know the bank's real costs, so we repeated the search for other ratios (Table 7). The cutoff moves a lot, and each time it's close to the theory value 1 / (1 + ratio). A bank would use its own numbers; the model itself stays the same.

Table 7. Best cutoff for different cost ratios (logistic regression, test-set results).

Cost ratio (miss : false alarm)	Cutoff	Clients flagged	Recall	Precision
2 : 1	0.34	16%	43%	58%
3 : 1	0.23	25%	55%	49%
5 : 1	0.16	41%	71%	38%
10 : 1	0.11	76%	92%	27%

5.5 Does the model sort clients well?

We sorted the test clients into ten equal groups by predicted risk (Figure 9). The lowest-risk 10% default only **6%** of the time, and the highest-risk 10% default **67%** of the time. The **top 20% of clients by predicted risk include 49% of all defaulters**. The model's average predicted probability in each group is close to the actual default rate, so the probabilities can be used about as they are. The biggest gap is in the riskiest group (predicted 0.73, actual 0.67).

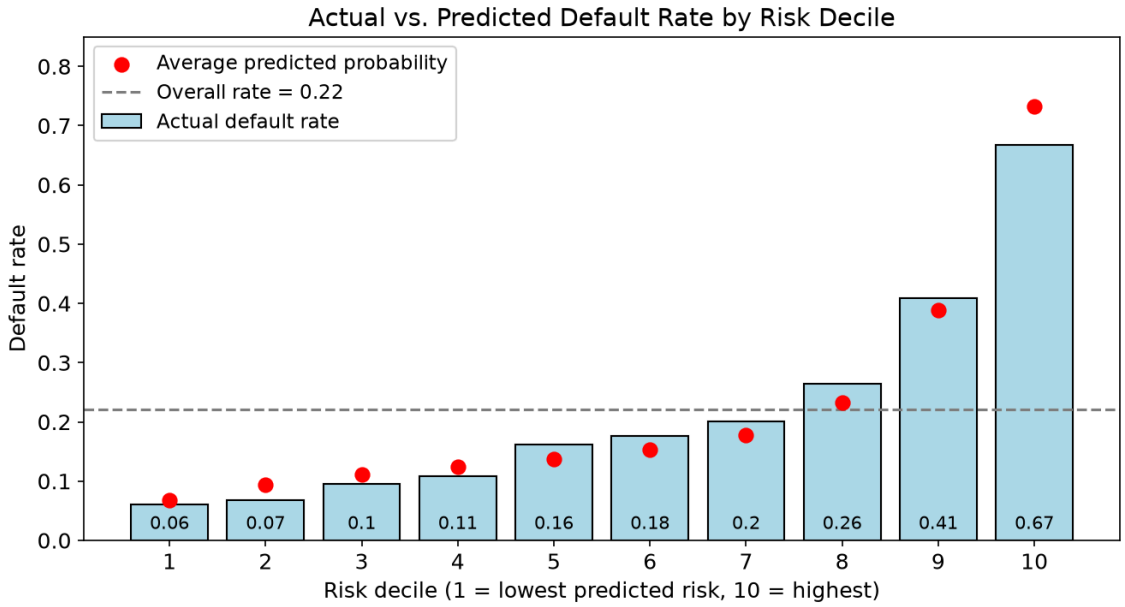


Figure 9. Actual default rate (bars) and average predicted probability (dots) for each risk decile (logistic regression, test set).

6. Conclusion and Recommendations

6.1 Conclusions

1. **How a client pays matters much more than who the client is.** Recent lateness is the strongest warning sign (odds ratio about 6 for 2+ months late in September), followed by a history of late months, low payments, high utilization and a low credit limit.
2. **Demographics add almost nothing.** Sex, education, marital status and age are statistically significant because the sample is large, but their effects are small. Taking out sex, marital status and age changes the test AUC by only 0.0001.
3. **A simple model is almost as good as a complex one.** Logistic regression with good features got a test AUC of 0.764, vs. 0.773 for the random forest. Our feature work mattered more than the choice of model.
4. **The cutoff matters as much as the model.** A cost-based cutoff of 0.16 doubles the share of defaulters caught and lowers the expected cost by about 25% compared with 0.5.

6.2 Recommendations for a bank

- **Use the logistic regression as the main model.** It gives up only about 0.01 AUC and 1.4% in expected cost compared with the random forest, but every prediction can be explained ("two months late in September multiplies the odds by about 6"). Like the Bayesian network question in Module 5, a model the bank can explain is easier to check, explain to clients and regulators, and act on. Keep the random forest as a benchmark.
- **Watch payment behavior every month.** A missed or late payment should trigger early, low-cost contact like a reminder or a payment-plan offer, before a second month of lateness.
- **Act in tiers.** Clients in the top risk decile (67% default rate) should get stronger steps, like a personal call or holding limit increases. Clients above the cost-based cutoff get reminders. Low-risk clients are left alone.
- **Set the cutoff from real costs.** Our 5 : 1 ratio is an assumption. The bank should estimate the real cost of a missed default and of a false alarm, and recompute the cutoff (Table 7).
- **Leave out sex, marital status and age.** They barely help the predictions and could raise fairness or legal problems.

6.3 Limitations and future work

- **Old, local data.** The data is from one bank in Taiwan in 2005, during a credit card debt crisis there. Default patterns today, or in other countries, may be different.
- **Short history.** Six months of history and one outcome month limit what the model can learn about longer-term behavior.
- **Assumed costs.** The 5 : 1 cost ratio is our assumption, not the bank's real number.

- **Unclear codes.** The meaning of repayment codes -2 and 0, and of the undocumented education and marital codes, comes from other studies, not from the data provider.
- **Future work:** add income or other financial data if we can get it, test on more recent data, and check fairness directly (for example, compare recall for men and women).

Team Contributions

This was a team effort. We met regularly, shared ideas, and made the main analysis decisions together. **Abhineet Sood** set up the repository and most of the code: data loading, cleaning, feature engineering, and the modeling and evaluation notebooks. **Ravi Kanth Vuddagiri** led the technical report and the video presentation. We both reviewed each other's work.

References

Agresti, A., & Kateri, M. (2022). *Foundations of statistics for data scientists: With R and Python*. CRC Press.

Elkan, C. (2001). The foundations of cost-sensitive learning. In *Proceedings of the Seventeenth International Joint Conference on Artificial Intelligence* (pp. 973–978). Morgan Kaufmann.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Seabold, S., & Perktold, J. (2010). Statsmodels: Econometric and statistical modeling with Python. In *Proceedings of the 9th Python in Science Conference* (pp. 92–96).

Yeh, I.-C. (2009). *Default of credit card clients* [Data set]. UCI Machine Learning Repository. https://doi.org/10.24432/C55S3H

Yeh, I.-C., & Lien, C.-H. (2009). The comparisons of data mining techniques for the predictive accuracy of probability of default of credit card clients. *Expert Systems with Applications, 36*(2), 2473–2480. https://doi.org/10.1016/j.eswa.2007.12.020

Use of Generative AI Tools — Disclosure

Tool used: Claude (Anthropic), accessed through Claude Code.

Purpose of use: We used Claude as a coding and writing partner for this project.

What we implemented and modified: We planned the analysis and made the decisions together (how to treat the undocumented codes, which features to build, which model to recommend, and the cost assumption). We used AI's help to set up the repository, fix Python syntax, and polish the wording of this report. We ran every notebook ourselves and checked that each number in the report matches the notebook output.

Appendix: Notebook Output

The following pages have the executed notebooks from the GitHub repository, in order:

- A. `01_data_prep_eda.ipynb`: loading, quality checks, cleaning, first look at the data
- B. `02_eda_statistical_tests.ipynb`: chi-squared tests, z test, t tests, correlation, VIF
- C. `03_modeling.ipynb`: features, logistic regression, likelihood ratio test, model comparison
- D. `04_evaluation.ipynb`: test-set results, bootstrap, cost-based cutoff, risk deciles

01 — Data Preparation and First Look

AAI-500 Final Team Project, Group 02 — Predicting Credit Card Default

Names: Abhineet Sood, Ravi Kanth Vuddagiri Date: October 4, 2026

Use of Generative AI Tools — Disclosure

Tool used: Claude (Anthropic), accessed through Claude Code.

Purpose of use: We used Claude as a coding and writing partner for the project.

What we implemented and modified: We planned the analysis and made the decisions together (how to clean the data, which features to build, which model to recommend). We used AI's help to set up the repository, fix Python syntax, and polish the wording. We ran every cell in these notebooks ourselves and checked that the numbers in the report match the outputs.

In this notebook we load the UCI *Default of Credit Card Clients* data, check it for problems, clean it with the steps in `src/cleaning.py`, and save a clean copy to `data/processed/`. At the end we take a first look at the target and a few predictors. Notebook 02 has the hypothesis tests.

In [1]:
```
import sys
from pathlib import Path

import matplotlib.pyplot as plt
import pandas as pd

# so the notebook can find our src/ folder
PROJECT_ROOT = Path.cwd().resolve().parent if Path.cwd().name ==
"notebooks" else Path.cwd().resolve()
sys.path.insert(0, str(PROJECT_ROOT))

from src.data_loader import load_raw, PROCESSED_DIR
from src import cleaning

# make all the graphs a bit bigger and easier to read
plt.rcParams.update({"figure.dpi": 110, "font.size": 12,
"axes.titlesize": 14,
                     "axes.labelsize": 12, "legend.fontsize": 11})
pd.set_option("display.max_columns", 30)

FIG_DIR = PROJECT_ROOT / "reports" / "figures"
FIG_DIR.mkdir(parents=True, exist_ok=True)
```

1. Load the raw data

The loader downloads the file from UCI the first time and saves it in `data/raw/`.

In [2]:
```
raw = load_raw()
print("Rows =", raw.shape[0], " Columns =", raw.shape[1])
raw.head()
```

```
Rows = 30000  Columns = 25
```

Out[2]:

	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_
0	1	20000	2	2	1	24	2	2	-1	
1	2	120000	2	2	2	26	-1	2	0	
2	3	90000	2	2	2	34	0	0	0	
3	4	50000	2	2	1	37	0	0	0	
4	5	50000	1	2	1	57	-1	0	-1	

There are **30,000 clients** and **25 columns**: an ID, 23 predictors and the target (`default payment next month`).

2. Data quality checks

In [3]:
```
raw.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 30000 entries, 0 to 29999
Data columns (total 25 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   ID                          30000 non-null  int64
 1   LIMIT_BAL                   30000 non-null  int64
 2   SEX                         30000 non-null  int64
 3   EDUCATION                   30000 non-null  int64
 4   MARRIAGE                    30000 non-null  int64
 5   AGE                         30000 non-null  int64
 6   PAY_0                       30000 non-null  int64
 7   PAY_2                       30000 non-null  int64
 8   PAY_3                       30000 non-null  int64
 9   PAY_4                       30000 non-null  int64
 10  PAY_5                       30000 non-null  int64
 11  PAY_6                       30000 non-null  int64
 12  BILL_AMT1                   30000 non-null  int64
 13  BILL_AMT2                   30000 non-null  int64
 14  BILL_AMT3                   30000 non-null  int64
 15  BILL_AMT4                   30000 non-null  int64
 16  BILL_AMT5                   30000 non-null  int64
 17  BILL_AMT6                   30000 non-null  int64
 18  PAY_AMT1                    30000 non-null  int64
 19  PAY_AMT2                    30000 non-null  int64
 20  PAY_AMT3                    30000 non-null  int64
 21  PAY_AMT4                    30000 non-null  int64
 22  PAY_AMT5                    30000 non-null  int64
 23  PAY_AMT6                    30000 non-null  int64
 24  default payment next month  30000 non-null  int64
dtypes: int64(25)
memory usage: 5.7 MB
```

In [4]:
```
print("Missing values =", raw.isna().sum().sum())
print("Duplicate rows (ignoring ID) =",
raw.drop(columns="ID").duplicated().sum())
```

```
Missing values = 0
Duplicate rows (ignoring ID) = 35
```

There are **no missing values**, but **35 rows** are exact copies of another row once we ignore the ID.

Category codes vs. the documentation

The UCI documentation lists EDUCATION as 1–4 and MARRIAGE as 1–3, and repayment status (`PAY_*`) as -1 = paid on time and 1–9 = months late. We check what is actually in the data.

In [5]:
```
for col in ["EDUCATION", "MARRIAGE", "PAY_0"]:
    print(col)
    print(raw[col].value_counts().sort_index().to_string())
    print()
```

```
EDUCATION
EDUCATION
0       14
1    10585
2    14030
3     4917
4      123
5      280
6       51

MARRIAGE
MARRIAGE
0       54
1    13659
2    15964
3      323

PAY_0
PAY_0
-2     2759
-1     5686
 0    14737
 1     3688
 2     2667
 3      322
 4       76
 5       26
 6       11
 7        9
 8       19
```

A few codes aren't in the documentation:

- EDUCATION has 0, 5 and 6 (**345 rows**), and MARRIAGE has 0 (**54 rows**).
- Most repayment values are **-2 or 0**, which the documentation doesn't explain. Other studies of this data read -2 as "no balance / card not used" and 0 as "revolving credit" (paid the minimum, not late). These are real account states, so we keep them.

Our cleaning decisions (also in the report):

Issue	Rows	Action	Why
`ID` column	all	Drop	It's just a row number
Exact duplicates (ignoring ID)	35	Drop	Would count the same client twice
`EDUCATION` = 0, 5, 6	345	Change to 4 ("Other")	Undocumented and rare
`MARRIAGE` = 0	54	Change to 3 ("Other")	Undocumented and rare
`PAY_*` = -2, 0	many	Keep	Real states (no balance / revolving)
`PAY_0` name	n/a	Rename to `PAY_1`	Lines up with `BILL_AMT1` and `PAY_AMT1`
Target name	n/a	Rename to `DEFAULT`	Shorter

3. Clean and save

In [6]:
```
df = cleaning.clean(raw)
print("Rows =", df.shape[0], " Columns =", df.shape[1])
print()
for col in ["EDUCATION", "MARRIAGE"]:
    print(col, df[col].value_counts().sort_index().to_dict())
```

```
Dropped 35 duplicate rows
Rows = 29965  Columns = 24

EDUCATION {1: 10563, 2: 14019, 3: 4915, 4: 468}
MARRIAGE {1: 13643, 2: 15945, 3: 377}
```

In [7]:
```
PROCESSED_DIR.mkdir(parents=True, exist_ok=True)
out_path = PROCESSED_DIR / "credit_default_clean.csv"
df.to_csv(out_path, index=False)
print("Saved", out_path.relative_to(PROJECT_ROOT))
```

```
Saved data/processed/credit_default_clean.csv
```

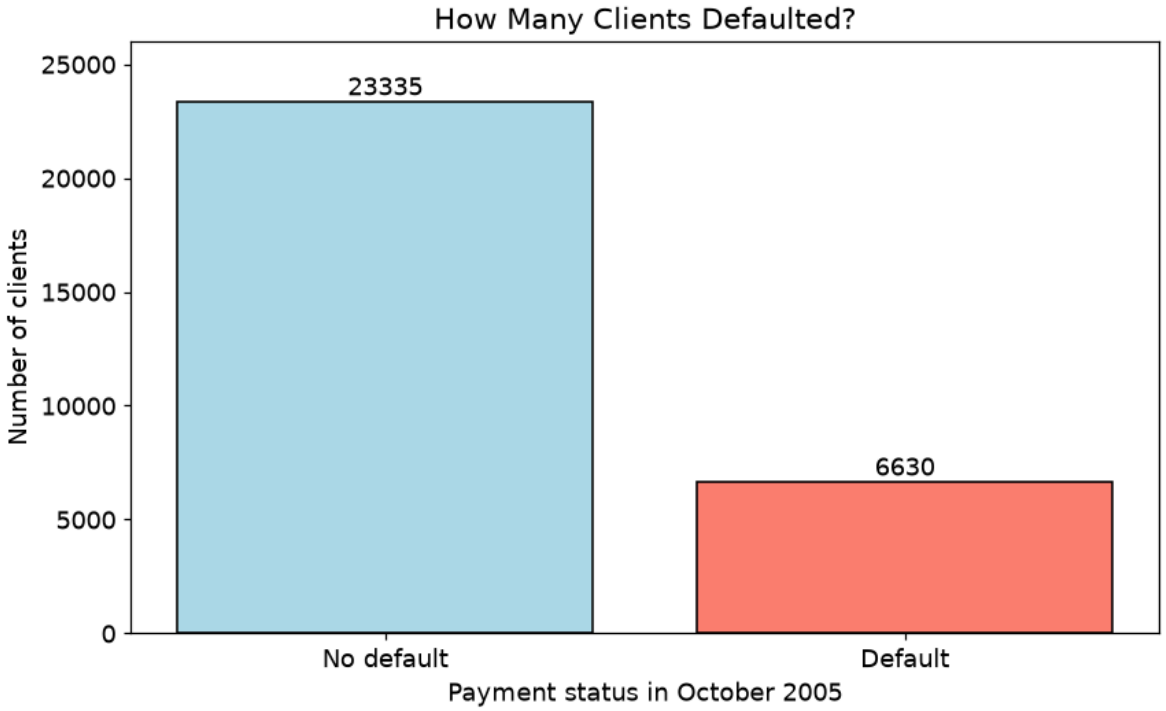
The clean data has **29,965 clients and 24 columns**. Notebooks 02–04 read this file.

4. First look: the target

In [8]:
```
counts = df["DEFAULT"].value_counts().sort_index()
rate = df["DEFAULT"].mean()
print("Did not default =", counts[0])
print("Defaulted =", counts[1])
print("Default rate =", round(rate, 4))

plt.figure(figsize=(8, 5))
plt.bar(["No default", "Default"], counts.values, color=["lightblue",
"salmon"], edgecolor="black")
for i, c in enumerate(counts.values):
    plt.text(i, c + 300, str(c), ha="center", fontsize=12)
plt.ylim(0, 26000)
plt.xlabel("Payment status in October 2005")
plt.ylabel("Number of clients")
plt.title("How Many Clients Defaulted?")
plt.tight_layout()
plt.savefig(FIG_DIR / "target_balance.png", dpi=150)
plt.show()
```

```
Did not default = 23335
Defaulted = 6630
Default rate = 0.2213
```

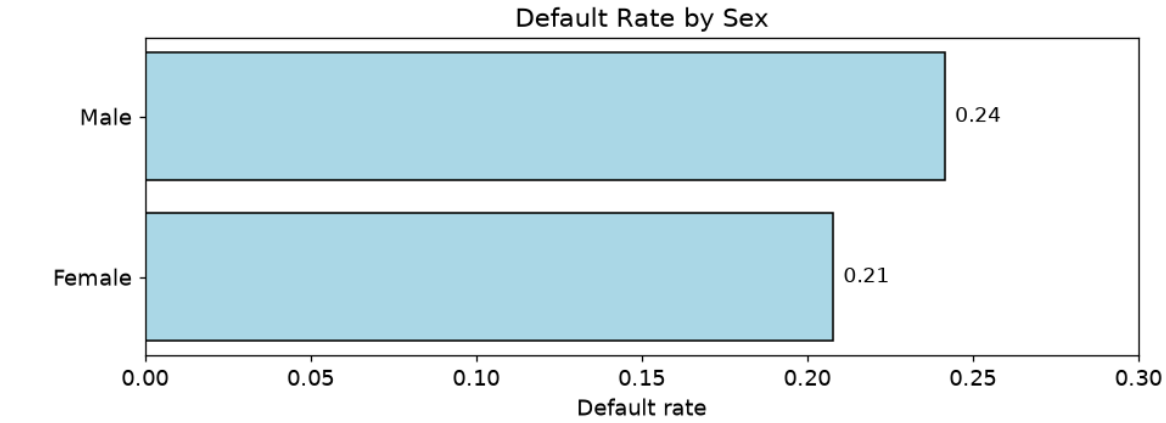


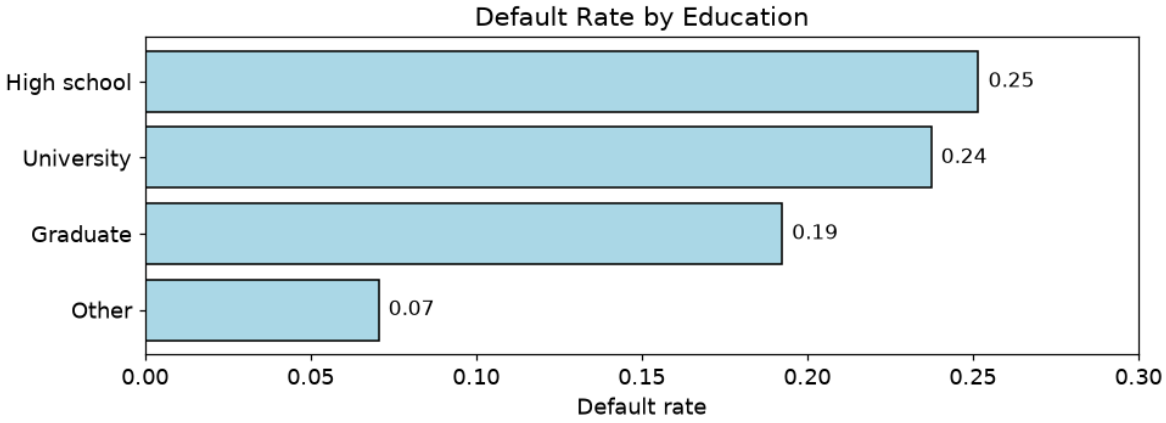
6,630 of 29,965 clients (22.13%) defaulted. The two groups are not balanced. A model that always says "no default" would already be about 78% accurate, so accuracy alone would be misleading. When we evaluate models we look at ROC-AUC, precision and recall instead.

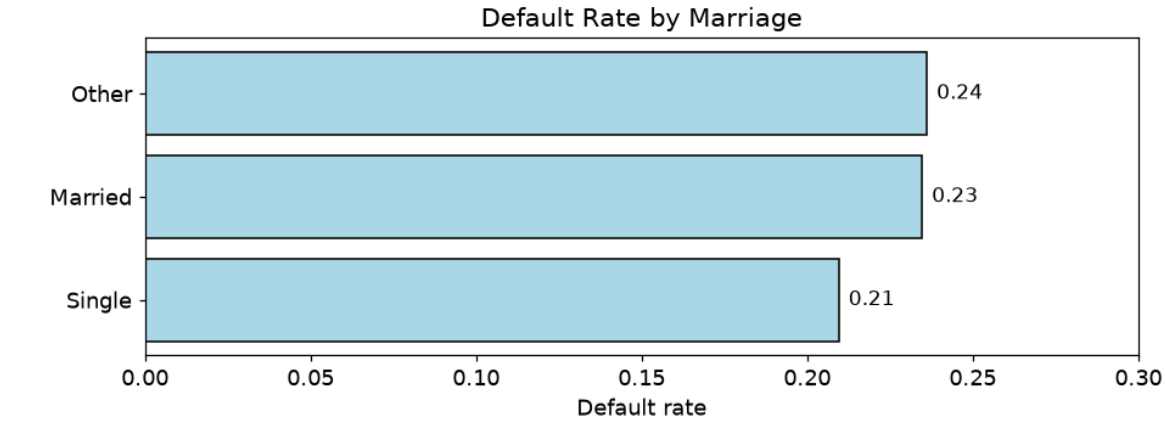
5. First look: default rate by demographics

In [9]:
```
labeled = cleaning.add_labels(df)

fig, axes = plt.subplots(3, 1, figsize=(9, 10))
for ax, col in zip(axes, ["SEX", "EDUCATION", "MARRIAGE"]):
    rates = labeled.groupby(col)["DEFAULT"].mean().sort_values()
    ax.barh(rates.index, rates.values, color="lightblue",
edgecolor="black")
    for i, v in enumerate(rates.values):
        ax.text(v + 0.003, i, str(round(v, 2)), va="center",
fontsize=11)
    ax.set_xlim(0, 0.3)
    ax.set_xlabel("Default rate")
    ax.set_title("Default Rate by " + col.title())
plt.tight_layout()
plt.savefig(FIG_DIR / "default_rate_by_demographics.png", dpi=150)
plt.show()
```


Default Rate by Sex
Male
Female
0.24
0.21
0.00
0.05
0.10
0.15
0.20
0.25
0.30
Default rate


Default Rate by Education
High school
University
Graduate
Other
0.25
0.24
0.19
0.07
0.00
0.05
0.10
0.15
0.20
0.25
0.30
Default rate


Default Rate by Marriage
Other
Married
Single
0.24
0.23
0.21
0.00
0.05
0.10
0.15
0.20
0.25
0.30
Default rate

The default rates are fairly close across groups: men **0.24** vs. women **0.21**, and married vs. single **0.23** vs. **0.21**. Education ranges from 0.19 (graduate school) to 0.25 (high school). The "Other" education group is much lower (0.07), but it's small and includes the undocumented codes we merged. None of these gaps look very big. Notebook 02 tests whether they are real.

6. First look: most recent repayment status

In [10]:
```
pay1 = df.groupby("PAY_1")["DEFAULT"].agg(["mean", "size"])
pay1.columns = ["default_rate", "clients"]
pay1.round(2)
```

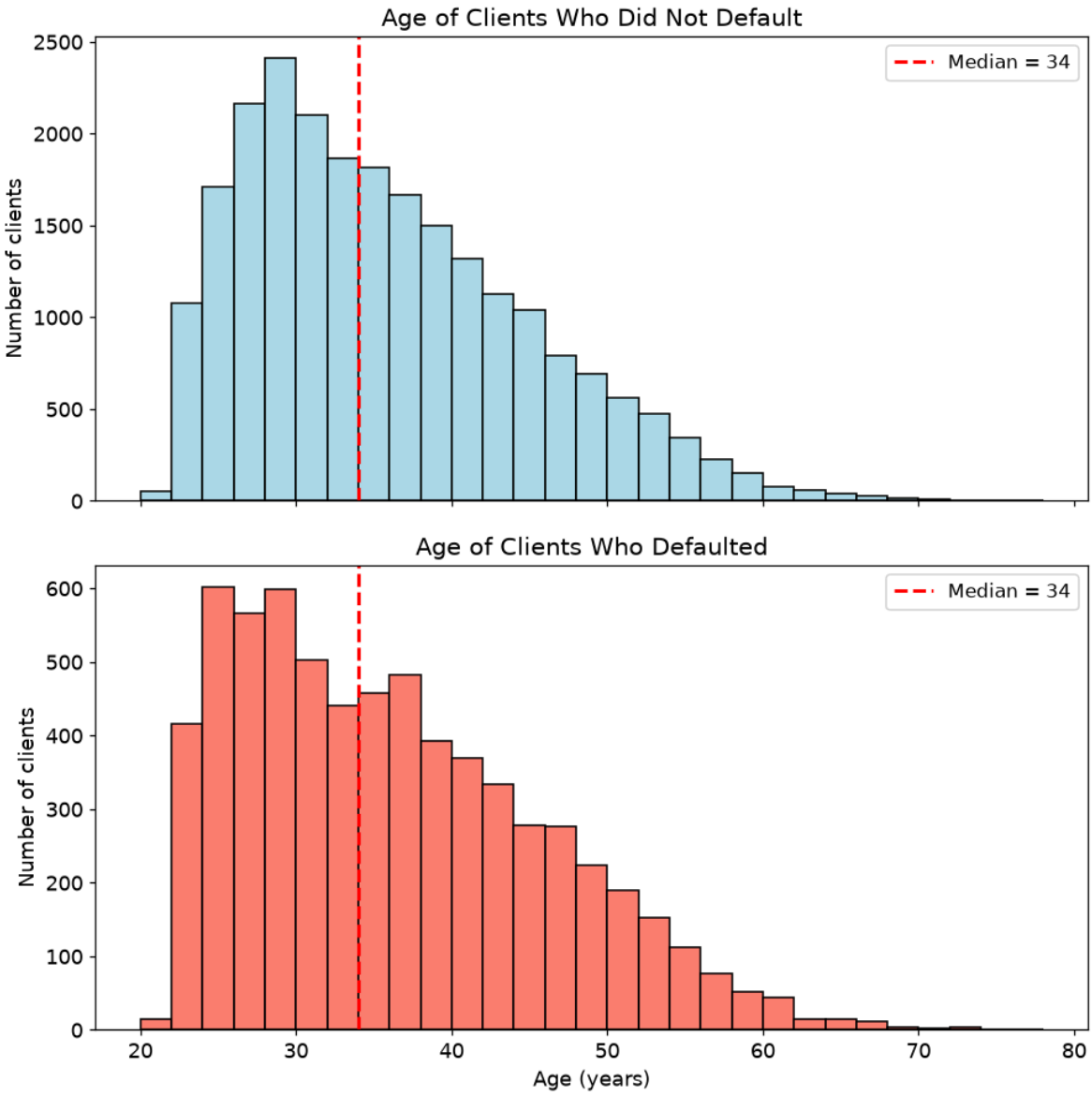
Out[10]:

PAY_1	default_rate	clients
-2	0.13	2750
-1	0.17	5682
0	0.13	14737
1	0.34	3667
2	0.69	2666
3	0.76	322
4	0.68	76
5	0.50	26
6	0.55	11
7	0.78	9
8	0.58	19

This is a much bigger pattern. Clients who paid on time or had no balance in September (-2, -1, 0) defaulted only **13–17%** of the time. Clients who were **1 month late** defaulted **34%** of the time, and clients **2 months late** defaulted **69%** of the time. The codes above 3 have very few clients, so their rates jump around.

7. First look: age

In [11]:
```
fig, axes = plt.subplots(2, 1, figsize=(9, 9), sharex=True)
groups = [(0, "Age of Clients Who Did Not Default", "lightblue"),
          (1, "Age of Clients Who Defaulted", "salmon")]
for ax, (value, title, color) in zip(axes, groups):
    ages = df.loc[df["DEFAULT"] == value, "AGE"]
    ax.hist(ages, bins=range(20, 80, 2), color=color, edgecolor="black")
    ax.axvline(ages.median(), color="red", linestyle="--", linewidth=2,
               label="Median = " + str(int(ages.median())))
    ax.set_ylabel("Number of clients")
    ax.set_title(title)
    ax.legend(loc="upper right")
axes[1].set_xlabel("Age (years)")
plt.tight_layout()
plt.savefig(FIG_DIR / "age_by_default.png", dpi=150)
plt.show()
```



Both age distributions are skewed right (most clients are in their late 20s and 30s) and have the **same median of 34**. Age doesn't look like it matters much.

Next steps

- **02_eda_statistical_tests:** chi-square tests, t-tests and confidence intervals for each variable, plus correlation and VIF.
- **03_modeling:** 70/30 train/test split, logistic regression with odds ratios, and comparison models.
- **04_evaluation:** test-set results, bootstrap confidence intervals, and choosing a decision threshold.

02 — EDA and Hypothesis Tests

AAI-500 Final Team Project, Group 02 — Predicting Credit Card Default

Names: Abhineet Sood, Ravi Kanth Vuddagiri Date: October 4, 2026

In notebook 01 we cleaned the data and took a first look. Here we test which variables are actually related to default:

- **Categorical variables** (sex, education, marital status, age group, repayment status): chi-squared test of independence.
- **Sex:** two-proportion z test and a 95% CI for the difference in default rates.
- **Numeric variables** (credit limit, age, bill and payment amounts): two-sample t test with `equal_var=False` and a 95% CI for the difference in means.
- **Correlation and VIF**, to find columns that carry the same information before we build models.

We use α = 0.05. With almost 30,000 clients, even tiny differences can come out significant, so for each test we also look at **how big** the difference is.

In [1]:
```
import sys
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

PROJECT_ROOT = Path.cwd().resolve().parent if Path.cwd().name ==
"notebooks" else Path.cwd().resolve()
sys.path.insert(0, str(PROJECT_ROOT))

from src.data_loader import load_raw, PROCESSED_DIR
from src import cleaning

plt.rcParams.update({"figure.dpi": 110, "font.size": 12,
"axes.titlesize": 14,
                     "axes.labelsize": 12, "legend.fontsize": 11})
pd.set_option("display.max_columns", 30)
FIG_DIR = PROJECT_ROOT / "reports" / "figures"
```

1. Load the clean data

Notebook 01 saves `credit_default_clean.csv`. If it isn't there yet, we rebuild it from the raw file.

In [2]:
```
clean_path = PROCESSED_DIR / "credit_default_clean.csv"
if clean_path.exists():
    df = pd.read_csv(clean_path)
else:
    df = cleaning.clean(load_raw())

labeled = cleaning.add_labels(df)
print("Rows =", df.shape[0], " Columns =", df.shape[1])
print("Default rate =", round(df["DEFAULT"].mean(), 4))
```

```
Rows = 29965  Columns = 24
Default rate = 0.2213
```

2. Summary statistics by default status

In [3]:
```
num_cols = ["LIMIT_BAL", "AGE", "BILL_AMT1", "PAY_AMT1", "PAY_AMT2"]
summary = df.groupby("DEFAULT")[num_cols].agg(["mean", "median",
"std"]).T.round(2)
summary.columns = ["No default", "Default"]
summary
```

Out[3]:

		No default	Default
LIMIT_BAL	**mean**	178084.51	129984.57
	median	150000.00	90000.00
	std	131665.89	115258.62
AGE	**mean**	35.42	35.72
	median	34.00	34.00
	std	9.08	9.70
BILL_AMT1	**mean**	52058.72	48552.81
	median	23242.00	20209.50
	std	73600.55	73801.17
PAY_AMT1	**mean**	6315.05	3400.12
	median	2474.00	1641.00
	std	18024.37	9548.02
PAY_AMT2	**mean**	6648.59	3391.72
	median	2253.00	1542.00
	std	25316.93	11742.85

Clients who defaulted had **lower credit limits** (median NTD 90,000 vs. NTD 150,000) and **paid less** in the last two months. Age and bill size look almost the same in both groups. For the dollar columns the mean is much bigger than the median, so these are **skewed right**: a few clients with very large values pull the mean up.

3. Chi-squared tests for the categorical variables

For each variable:

- H_0: the variable and default are independent (the default rate is the same in every group)
- H_a: the default rate is different in at least one group

For September repayment status we group the codes into four levels. Codes 3–8 have fewer than 500 clients combined, so they go into "2+ months late". This also keeps every expected count above 5, which the chi-squared test needs. For age we use five age groups.

In [4]:

```
from scipy import stats

def sep_status_group(pay1):
    if pay1 <= -1:
        return "Paid / no balance"
    if pay1 == 0:
        return "Revolving"
    if pay1 == 1:
        return "1 month late"
    return "2+ months late"

labeled["SEP_STATUS"] = df["PAY_1"].apply(sep_status_group)
labeled["AGE_GROUP"] = pd.cut(df["AGE"], bins=[20, 30, 40, 50, 60, 80],
                              labels=["21-30", "31-40", "41-50", "51-
60", "61+"])

for col in ["SEX", "EDUCATION", "MARRIAGE", "AGE_GROUP", "SEP_STATUS"]:
    table = pd.crosstab(labeled[col], labeled["DEFAULT"])
    chi2 = stats.chi2_contingency(table)
    rates = labeled.groupby(col, observed=True)["DEFAULT"].mean()

    print(col)
    print("  Chi-squared =", round(chi2.statistic, 2), " df =",
chi2.dof, " P-value =", chi2.pvalue)
    print("  Smallest expected count =", round(chi2.expected_freq.min(),
2))
    print("  Default rate by group:", rates.round(2).to_dict())
    print("  Gap between highest and lowest rate =", round(rates.max() -
rates.min(), 2))
    print()
```

```
SEX
  Chi-squared = 47.13  df = 1  P-value = 6.639288761646278e-12
  Smallest expected count = 2627.22
  Default rate by group: {'Female': 0.21, 'Male': 0.24}
  Gap between highest and lowest rate = 0.03

EDUCATION
  Chi-squared = 160.46  df = 3  P-value = 1.458089270856381e-34
  Smallest expected count = 103.55
  Default rate by group: {'Graduate': 0.19, 'High school': 0.25, 'Other':
0.07, 'University': 0.24}
  Gap between highest and lowest rate = 0.18

MARRIAGE
  Chi-squared = 27.49  df = 2  P-value = 1.073731946266748e-06
  Smallest expected count = 83.41
  Default rate by group: {'Married': 0.23, 'Other': 0.24, 'Single': 0.21}
  Gap between highest and lowest rate = 0.03

AGE_GROUP
  Chi-squared = 37.49  df = 4  P-value = 1.4288631846501155e-07
  Smallest expected count = 60.18
  Default rate by group: {'21-30': 0.22, '31-40': 0.2, '41-50': 0.23, '51-
60': 0.25, '61+': 0.27}
  Gap between highest and lowest rate = 0.06

SEP_STATUS
  Chi-squared = 5333.29  df = 3  P-value = 0.0
  Smallest expected count = 692.32
  Default rate by group: {'1 month late': 0.34, '2+ months late': 0.7, 'Pa
id / no balance': 0.16, 'Revolving': 0.13}
  Gap between highest and lowest rate = 0.57
```

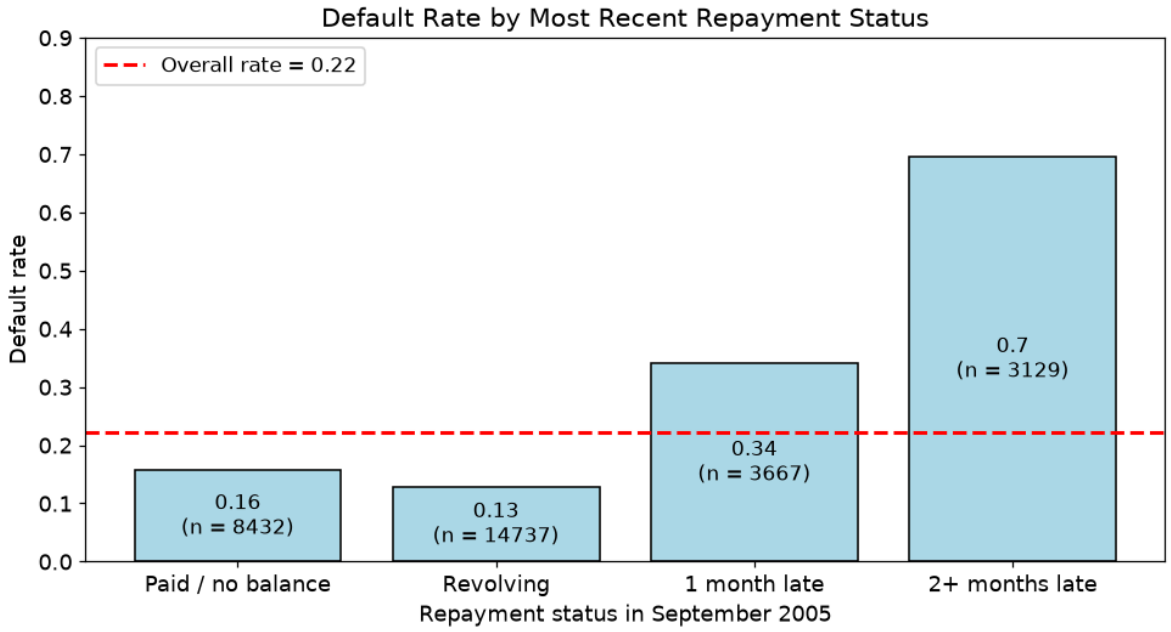
All five P-values are below 0.001, so we **reject H_0** for every variable. But the size of the gaps is very different:

- **September repayment status:** the default rate goes from **0.13** (revolving) to **0.70** (2+ months late), a gap of **0.57**. This is a huge difference.
- **Sex, marital status and age group:** the gaps are only about **0.03 to 0.06**. These come out significant mostly because the sample is so large.
- **Education:** the gap is 0.18, but that's because of the small "Other" group (468 clients, many with undocumented codes). Without it, the rates only go from 0.19 to 0.25.

All the smallest expected counts are well above 5, so the chi-squared test is fine to use here.

In [5]:
```
order = ["Paid / no balance", "Revolving", "1 month late", "2+ months
late"]
rates = labeled.groupby("SEP_STATUS")["DEFAULT"].agg(["mean",
"size"]).loc[order]

plt.figure(figsize=(9, 5))
plt.bar(order, rates["mean"], color="lightblue", edgecolor="black")
for i, (rate, n) in enumerate(rates.values):
    plt.text(i, rate / 2, str(round(rate, 2)) + "\n(n = " + str(int(n))
+ ")",
             ha="center", va="center", fontsize=11)
plt.axhline(df["DEFAULT"].mean(), color="red", linestyle="--",
linewidth=2,
            label="Overall rate = " + str(round(df["DEFAULT"].mean(),
2)))
plt.ylim(0, 0.9)
plt.xlabel("Repayment status in September 2005")
plt.ylabel("Default rate")
plt.title("Default Rate by Most Recent Repayment Status")
plt.legend(loc="upper left")
plt.tight_layout()
plt.savefig(FIG_DIR / "default_rate_by_sep_status.png", dpi=150)
plt.show()
```



This is the strongest pattern in the data. Clients who paid in full or carried a normal revolving balance in September defaulted **13–16%** of the time. One month late raises that to **34%**, and two or more months late to **70%**.

4. Sex: two-proportion z test and 95% CI

Sex is the variable people worry about most for fairness, so we look at the size of the gap directly.

- H_0: π(male) = π(female)
- H_a: π(male) ≠ π(female)

In [6]:
```
from statsmodels.stats.proportion import proportions_ztest

male = df.loc[df["SEX"] == 1, "DEFAULT"]
female = df.loc[df["SEX"] == 2, "DEFAULT"]
z, p_value = proportions_ztest([male.sum(), female.sum()], [len(male),
len(female)])

p1 = male.mean()
p2 = female.mean()
diff = p1 - p2
se = np.sqrt(p1 * (1 - p1) / len(male) + p2 * (1 - p2) / len(female))

print("Male default rate =", round(p1, 4), " n =", len(male))
print("Female default rate =", round(p2, 4), " n =", len(female))
print("Difference (male - female) =", round(diff, 4))
print("z =", round(z, 2), " P-value =", p_value)
print("95% CI for the difference:", round(diff - 1.96 * se, 4), "to",
round(diff + 1.96 * se, 4))
```

```
Male default rate = 0.2416  n = 11874
Female default rate = 0.2079  n = 18091
Difference (male - female) = 0.0337
z = 6.88  P-value = 6.008917036223772e-12
95% CI for the difference: 0.024 to 0.0434
```

Men defaulted **24.16%** of the time and women **20.79%**. The P-value is basically 0, so the difference is real. The 95% CI says men default about **2.40 to 4.34 percentage points** more often. That's a small gap compared with the 20 to 50 point jumps we see with repayment status.

5. t tests for the numeric variables

For each numeric column:

- H_0: the mean is the same for clients who defaulted and clients who didn't
- H_a: the means are different

We use a two-sample t test with `equal_var=False` (the two groups have different spreads) and report the 95% CI for the difference (default – no default). The dollar columns are very skewed, so, like we did with the house prices in Module 4, we also run the same test on log(1 + amount) as a check.

In [7]:

```
for col in ["LIMIT_BAL", "AGE", "BILL_AMT1", "PAY_AMT1", "PAY_AMT2"]:
    yes = df.loc[df["DEFAULT"] == 1, col]
    no = df.loc[df["DEFAULT"] == 0, col]
    result = stats.ttest_ind(yes, no, equal_var=False)
    ci = result.confidence_interval(0.95)

    print(col)
    print("  Mean (default) =", round(yes.mean(), 2), " Mean (no
default) =", round(no.mean(), 2))
    print("  Difference =", round(yes.mean() - no.mean(), 2))
    print("  t =", round(result.statistic, 2), " P-value =",
result.pvalue)
    print("  95% CI for the difference:", round(ci.low, 2), "to",
round(ci.high, 2))
    if col != "AGE":
        # a few bills are negative (credits), so we set those to 0
before taking the log
        log_result = stats.ttest_ind(np.log1p(yes.clip(lower=0)),
np.log1p(no.clip(lower=0)), equal_var=False)
        print("  Same test on log(1 + amount): P-value =",
log_result.pvalue)
    print()
```

```
LIMIT_BAL
  Mean (default) = 129984.57  Mean (no default) = 178084.51
  Difference = -48099.94
  t = -29.02  P-value = 4.7916683365578764e-179
  95% CI for the difference: -51348.5 to -44851.38
  Same test on log(1 + amount): P-value = 2.212306212367346e-183

AGE
  Mean (default) = 35.72  Mean (no default) = 35.42
  Difference = 0.3
  t = 2.27  P-value = 0.023050035669612313
  95% CI for the difference: 0.04 to 0.56

BILL_AMT1
  Mean (default) = 48552.81  Mean (no default) = 52058.72
  Difference = -3505.91
  t = -3.42  P-value = 0.0006390717462602051
  95% CI for the difference: -5517.99 to -1493.82
  Same test on log(1 + amount): P-value = 2.495398819242878e-05

PAY_AMT1
  Mean (default) = 3400.12  Mean (no default) = 6315.05
  Difference = -2914.93
  t = -17.52  P-value = 2.971666206918408e-68
  95% CI for the difference: -3240.99 to -2588.87
  Same test on log(1 + amount): P-value = 2.769676056037276e-156

PAY_AMT2
  Mean (default) = 3391.72  Mean (no default) = 6648.59
  Difference = -3256.88
  t = -14.82  P-value = 1.6881160182607096e-49
  95% CI for the difference: -3687.49 to -2826.26
  Same test on log(1 + amount): P-value = 8.513364102808199e-128
```

- **Credit limit:** defaulters had limits about **NTD 48,100 lower** on average. The 95% CI is **NTD 44,851 to 51,349 lower**, and the P-value is basically 0.
- **Payments:** defaulters paid about **NTD 2,915 less in September** (95% CI 2,589 to 3,241 less) and **NTD 3,257 less in August**. That's roughly **half as much** as non-defaulters.
- **September bill:** defaulters' bills were about NTD 3,506 lower (95% CI 1,494 to 5,518), P = 0.0006. That's significant, but it's only about 7% of the average bill, so bill size alone doesn't tell us much.
- **Age:** P = 0.02, so it's significant at $\alpha = 0.05$. But the difference is only **0.30 years** (95% CI **0.04 to 0.56**), which is about 3 to 4 months. That's too small to matter in practice.

The log(1 + amount) checks give the same answers (all P-values close to 0), so the skew doesn't change our conclusions. Since the samples are very large (6,630 and 23,335), the t test works fine here even though the amounts aren't normal.

In [8]:

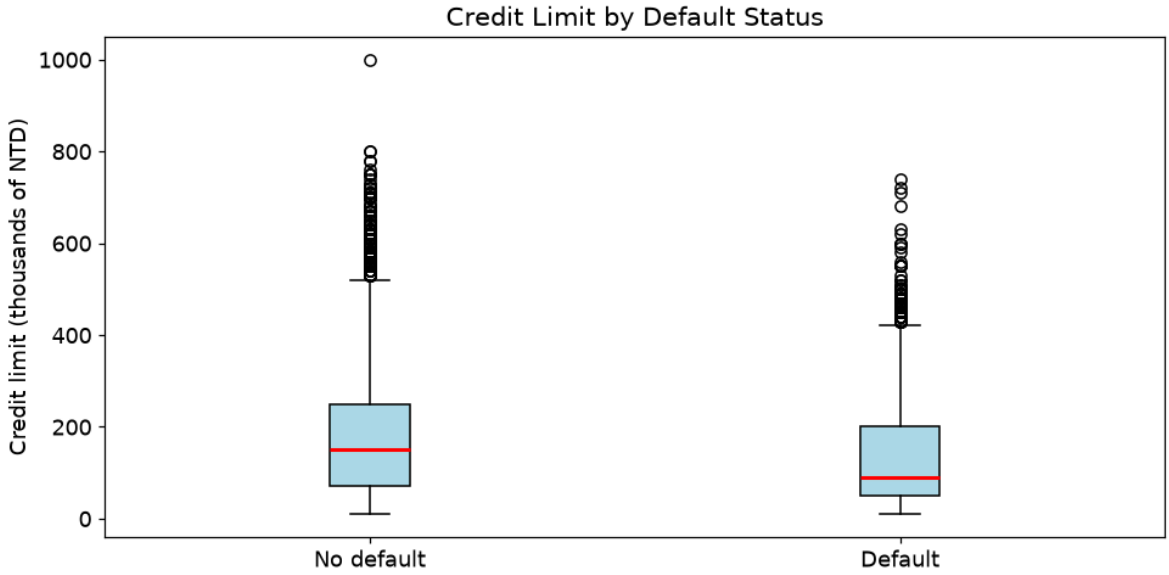
```
fig, axes = plt.subplots(2, 1, figsize=(9, 9))

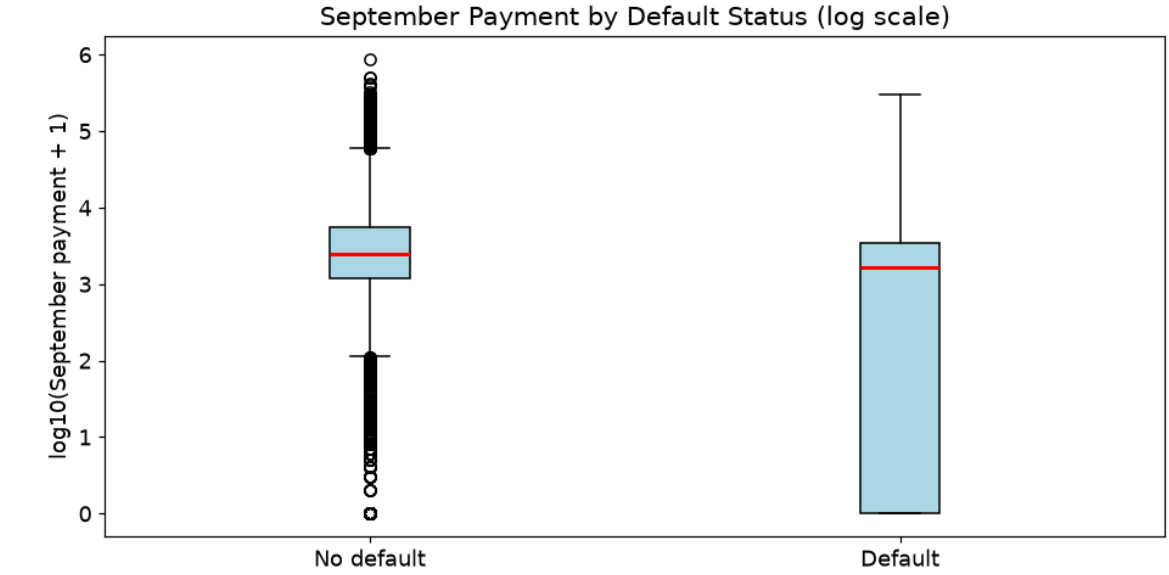
limit_no = df.loc[df["DEFAULT"] == 0, "LIMIT_BAL"] / 1000
limit_yes = df.loc[df["DEFAULT"] == 1, "LIMIT_BAL"] / 1000
axes[0].boxplot([limit_no, limit_yes], tick_labels=["No default",
"Default"], patch_artist=True,
                boxprops=dict(facecolor="lightblue"),
medianprops=dict(color="red", linewidth=2))
axes[0].set_ylabel("Credit limit (thousands of NTD)")
axes[0].set_title("Credit Limit by Default Status")

pay_no = np.log10(df.loc[df["DEFAULT"] == 0, "PAY_AMT1"] + 1)
pay_yes = np.log10(df.loc[df["DEFAULT"] == 1, "PAY_AMT1"] + 1)
axes[1].boxplot([pay_no, pay_yes], tick_labels=["No default",
"Default"], patch_artist=True,
                boxprops=dict(facecolor="lightblue"),
medianprops=dict(color="red", linewidth=2))
axes[1].set_ylabel("log10(September payment + 1)")
axes[1].set_title("September Payment by Default Status (log scale)")

plt.tight_layout()
plt.savefig(FIG_DIR / "limit_payment_by_default.png", dpi=150)
plt.show()

print("Share who paid 0 in September (default) =",
round((df.loc[df["DEFAULT"] == 1, "PAY_AMT1"] == 0).mean(), 2))
print("Share who paid 0 in September (no default) =",
round((df.loc[df["DEFAULT"] == 0, "PAY_AMT1"] == 0).mean(), 2))
```






```
Share who paid 0 in September (default) = 0.28
Share who paid 0 in September (no default) = 0.14
```

Both boxplots show what the tests found: the "Default" box sits lower. The payment plot uses a log scale because payments go from 0 to over NTD 800,000. Without it, the boxes would be squashed at the bottom. The bottom of the "Default" box reaches 0 because **28% of defaulters paid nothing at all in September**, compared with 14% of non-defaulters.

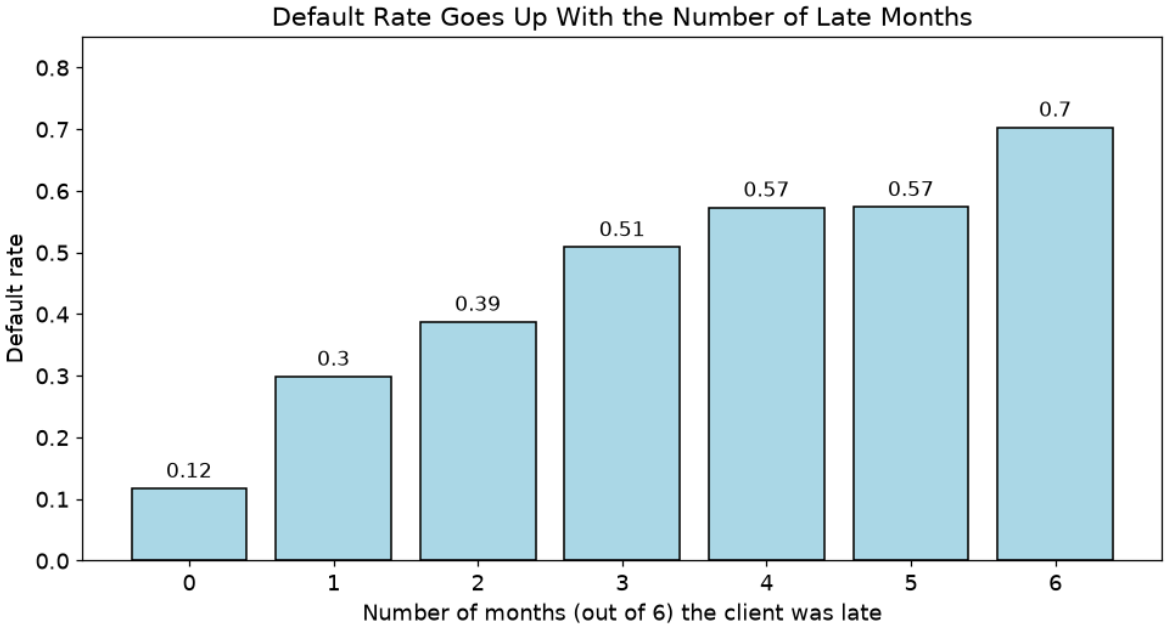
6. Payment history over six months

So far we only used September. Next we count how many of the six months each client was at least one month late.

In [9]:
```
months_late = (df[cleaning.PAY_STATUS_COLS] >= 1).sum(axis=1)
by_count = df.groupby(months_late)["DEFAULT"].agg(["mean", "size"])
by_count.columns = ["default_rate", "clients"]
print(by_count.round(2))

plt.figure(figsize=(9, 5))
plt.bar(by_count.index, by_count["default_rate"], color="lightblue",
edgecolor="black")
for x, rate in zip(by_count.index, by_count["default_rate"]):
    plt.text(x, rate + 0.01, str(round(rate, 2)), ha="center",
va="bottom", fontsize=11)
plt.xticks(range(7))
plt.ylim(0, 0.85)
plt.xlabel("Number of months (out of 6) the client was late")
plt.ylabel("Default rate")
plt.title("Default Rate Goes Up With the Number of Late Months")
plt.tight_layout()
plt.savefig(FIG_DIR / "default_rate_by_months_late.png", dpi=150)
plt.show()
```

```
   default_rate  clients
0          0.12    19918
1          0.30     4405
2          0.39     1899
3          0.51     1154
4          0.57      951
5          0.57      298
6          0.70     1340
```



The default rate goes up almost every step: **0.12** for clients who were never late, **0.30** after one late month, **0.51** after three, and **0.70** for clients late in all six months. A longer history of late payments is a strong warning sign, so our models will use both the most recent month and a count of the earlier late months.

7. Correlation between the predictors

If two predictors are very highly correlated, a logistic regression can't tell their effects apart and the coefficients become unstable. So we check this before modeling.

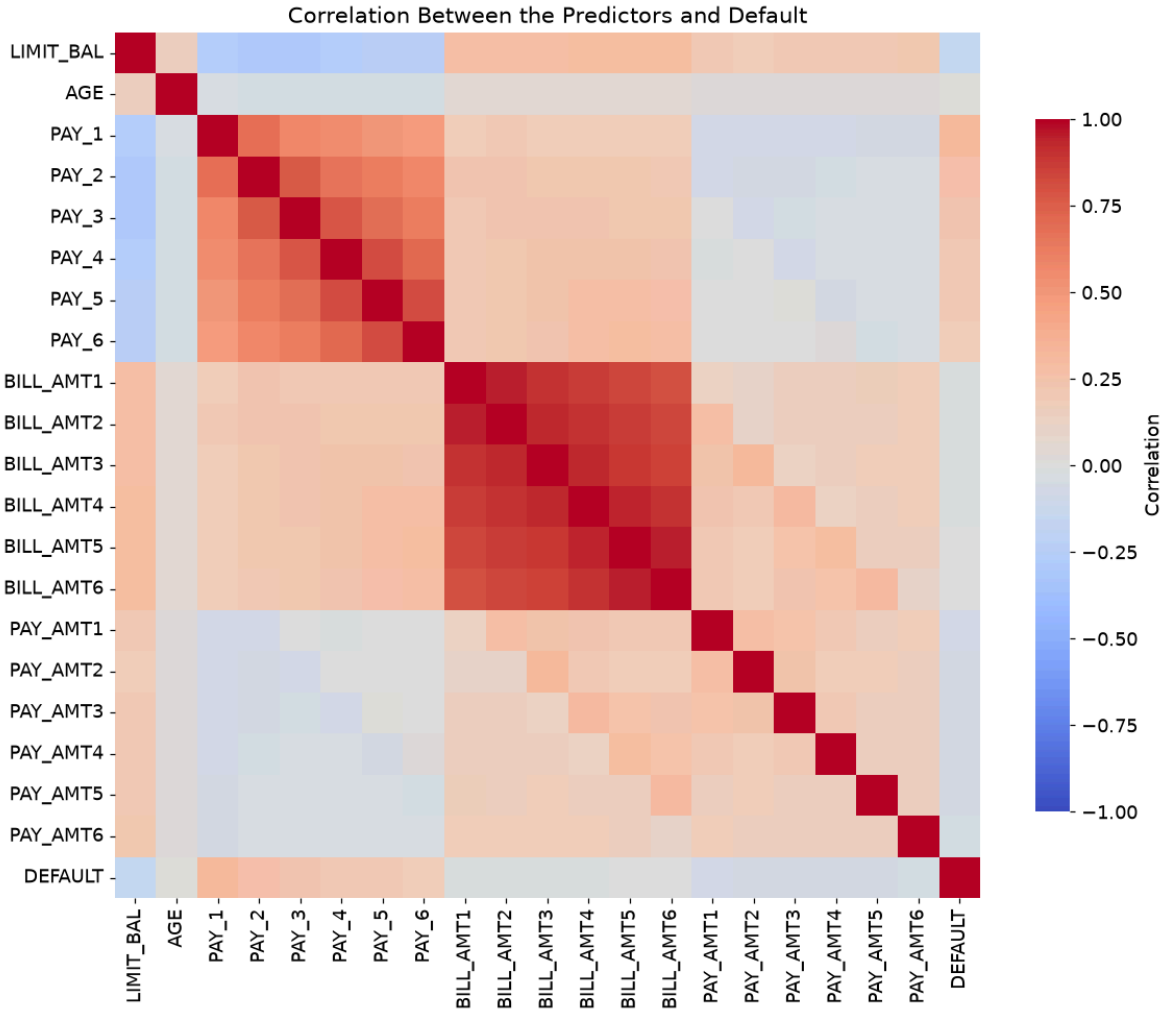
In [10]:

```
import seaborn as sns

corr_cols = (["LIMIT_BAL", "AGE"] + cleaning.PAY_STATUS_COLS +
cleaning.BILL_AMT_COLS
             + cleaning.PAY_AMT_COLS + ["DEFAULT"])
corr = df[corr_cols].corr()

plt.figure(figsize=(11, 9))
sns.heatmap(corr, cmap="coolwarm", vmin=-1, vmax=1, center=0,
square=True,
            cbar_kws={"label": "Correlation", "shrink": 0.8})
plt.title("Correlation Between the Predictors and Default")
plt.tight_layout()
plt.savefig(FIG_DIR / "correlation_heatmap.png", dpi=150)
plt.show()

print("Correlation with DEFAULT:")
print(corr["DEFAULT"].drop("DEFAULT").sort_values(ascending=False).round
(2).to_string())
```



```
Correlation with DEFAULT:
PAY_1        0.32
PAY_2        0.26
PAY_3        0.24
PAY_4        0.22
PAY_5        0.20
PAY_6        0.19
AGE          0.01
BILL_AMT6   -0.01
BILL_AMT5   -0.01
BILL_AMT4   -0.01
BILL_AMT3   -0.01
BILL_AMT2   -0.01
BILL_AMT1   -0.02
PAY_AMT6    -0.05
PAY_AMT5    -0.06
PAY_AMT3    -0.06
PAY_AMT4    -0.06
PAY_AMT2    -0.06
PAY_AMT1    -0.07
LIMIT_BAL   -0.15
```

Two blocks stand out:

- The six **bill amounts** are very highly correlated with each other (**0.80 to 0.95**). A client's balance doesn't change much from month to month.
- The six **repayment status** columns are also correlated (**0.48 to 0.82**).

The columns most correlated with default are the repayment status columns, led by September (**r = 0.32**). Credit limit has **r = -0.15**: higher limits go with less default.

Variance inflation factor (VIF) for the bill amounts

VIF shows how much a coefficient's variance is inflated because that column can be predicted from the other columns. A common rule of thumb is that a VIF above 5 to 10 is a problem.

In [11]:
```
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import
variance_inflation_factor

X_bill = sm.add_constant(df[cleaning.BILL_AMT_COLS])
for i, col in enumerate(cleaning.BILL_AMT_COLS, start=1):
    print(col, "VIF =", round(variance_inflation_factor(X_bill.values,
i), 2))
```

```
BILL_AMT1 VIF = 10.65
BILL_AMT2 VIF = 16.01
BILL_AMT3 VIF = 10.75
BILL_AMT4 VIF = 13.28
BILL_AMT5 VIF = 15.69
BILL_AMT6 VIF = 9.7
```

All six bill columns have VIF around **10 to 16**. If we put all six into a logistic regression, we couldn't trust their coefficients. In notebook 03 we keep only September's bill, divided by the credit limit (how much of the limit the client is using).

8. Summary

Variable	Test	Result	How big?
September repayment status	Chi-squared	P ≈ 0	**Huge** (0.13 → 0.70)
Months late (6-month history)	Default rate table	0.12 → 0.70	**Huge**
Credit limit	t test	P ≈ 0	About NTD 48,100 lower for defaulters
Recent payments	t test	P ≈ 0	Defaulters paid about half as much
Sex	Chi-squared, z test	P ≈ 0	Small (2.40 to 4.34 points)
Education, marital status, age group	Chi-squared	P < 0.001	Small
Age	t test	P = 0.02	Tiny (0.30 years)
Bill amount	t test	P = 0.0006	Small; highly correlated with other months

Takeaway: how a client has been paying matters much more than who the client is. Next, notebook 03 uses these findings to build a logistic regression and compare it with other models.

03 — Modeling: Logistic Regression and Comparison Models

AAI-500 Final Team Project, Group 02 — Predicting Credit Card Default

Names: Abhineet Sood, Ravi Kanth Vuddagiri Date: October 4, 2026

Our main model is a **logistic regression**. It gives each client a probability of default, and its coefficients can be explained as odds ratios with P-values and confidence intervals. We then compare it with three other models to see how much accuracy (if any) we give up by using a model we can explain.

Plan:

1. Split the data **70% train / 30% test**, keeping the 22% default rate in both parts.
2. Build features based on what we found in notebook 02.
3. Fit two logistic regressions (profile only vs. profile + payment behavior) and use a likelihood ratio test to see if payment behavior adds real information.
4. Compare four models with 5-fold cross-validation on the training set only.
5. Save the test-set predictions for notebook 04. The test set is not used to choose a model.

In [1]:
```
import sys
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

PROJECT_ROOT = Path.cwd().resolve().parent if Path.cwd().name ==
"notebooks" else Path.cwd().resolve()
sys.path.insert(0, str(PROJECT_ROOT))

from src.data_loader import load_raw, PROCESSED_DIR
from src import cleaning
from src.features import build_features, split, FEATURE_LABELS

plt.rcParams.update({"figure.dpi": 110, "font.size": 12,
"axes.titlesize": 14,
                     "axes.labelsize": 12, "legend.fontsize": 11})
pd.set_option("display.max_columns", 30)
FIG_DIR = PROJECT_ROOT / "reports" / "figures"
TABLE_DIR = PROJECT_ROOT / "reports" / "tables"
TABLE_DIR.mkdir(parents=True, exist_ok=True)
```

1. Load and split

We use a 70/30 split, like in Module 5. `stratify` keeps the default rate the same in both parts, and the fixed seed (500) means every notebook gets exactly the same rows.

In [2]:
```
clean_path = PROCESSED_DIR / "credit_default_clean.csv"
if clean_path.exists():
    df = pd.read_csv(clean_path)
else:
    df = cleaning.clean(load_raw())

train, test = split(df)
print("Training set: n =", len(train), " default rate =",
round(train["DEFAULT"].mean(), 4))
print("Test set:     n =", len(test), " default rate =",
round(test["DEFAULT"].mean(), 4))
```

```
Training set: n = 20975  default rate = 0.2213
Test set:     n = 8990  default rate = 0.2212
```

The training set has **20,975 clients** and the test set has **8,990**. Both have a default rate of about **0.22**, so the split worked the way we wanted.

2. Features

`src/features.py` builds 15 features for the logistic regression. Each choice comes from notebook 02:

Feature	Why
Credit limit per NTD 10,000, age per 10 years	Odds ratios per NTD 1 or per year would be too small to read
0/1 dummies for sex, education, marital status	Reference groups: male, graduate school, married
September status in 4 groups	Strongest predictor; rare codes 3–8 grouped as "2+ months late"
Count of late months, April–August	Sums up five correlated status columns into one
September bill / credit limit	Replaces six bill columns that had VIF 10–16
log(1 + payment) for September and August	Payment amounts are very skewed right

In [3]:
```
X_train = build_features(train)
X_test = build_features(test)
y_train = train["DEFAULT"]
y_test = test["DEFAULT"]
X_train.describe().T.round(2)
```

Out[3]:

	count	mean	std	min	25%	50%	75%	max
LIMIT_10K	20975.0	16.73	12.97	1.00	5.00	14.00	24.00	100.00
AGE_10Y	20975.0	3.55	0.92	2.10	2.80	3.40	4.20	7.50
FEMALE	20975.0	0.60	0.49	0.00	0.00	1.00	1.00	1.00
EDU_UNIV	20975.0	0.47	0.50	0.00	0.00	0.00	1.00	1.00
EDU_HS	20975.0	0.17	0.37	0.00	0.00	0.00	0.00	1.00
EDU_OTHER	20975.0	0.02	0.12	0.00	0.00	0.00	0.00	1.00
SINGLE	20975.0	0.53	0.50	0.00	0.00	1.00	1.00	1.00
MAR_OTHER	20975.0	0.01	0.11	0.00	0.00	0.00	0.00	1.00
SEP_REVOLVING	20975.0	0.49	0.50	0.00	0.00	0.00	1.00	1.00
SEP_LATE_1M	20975.0	0.12	0.32	0.00	0.00	0.00	0.00	1.00
SEP_LATE_2PLUS	20975.0	0.10	0.31	0.00	0.00	0.00	0.00	1.00
MONTHS_LATE_PRIOR	20975.0	0.60	1.28	0.00	0.00	0.00	1.00	5.00
UTILIZATION	20975.0	0.42	0.41	-0.62	0.02	0.32	0.83	3.00
LOG_PAY_AMT1	20975.0	6.64	3.25	0.00	6.91	7.65	8.52	13.68
LOG_PAY_AMT2	20975.0	6.58	3.27	0.00	6.75	7.61	8.52	14.34

Multicollinearity check

We check VIF again, this time on our final features.

In [4]:
```
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import
variance_inflation_factor

X_vif = sm.add_constant(X_train)
for i, col in enumerate(X_train.columns, start=1):
    print(col, "VIF =", round(variance_inflation_factor(X_vif.values,
i), 2))
```

```
LIMIT_10K VIF = 1.38
AGE_10Y VIF = 1.39
FEMALE VIF = 1.02
EDU_UNIV VIF = 1.38
EDU_HS VIF = 1.43
EDU_OTHER VIF = 1.04
SINGLE VIF = 1.33
MAR_OTHER VIF = 1.02
SEP_REVOLVING VIF = 2.18
SEP_LATE_1M VIF = 1.59
SEP_LATE_2PLUS VIF = 1.93
MONTHS_LATE_PRIOR VIF = 1.61
UTILIZATION VIF = 1.82
LOG_PAY_AMT1 VIF = 1.51
LOG_PAY_AMT2 VIF = 1.3
```

Every VIF is now **below 2.2**, well under the cutoff of 5. Replacing the six bill columns and the five older status columns with one summary each fixed the multicollinearity problem from notebook 02.

3. Logistic regression

The model is

$$\log\left(\frac{P(\text{default})}{1-P(\text{default})}\right) = \beta_0 + \beta_1 x_1 + \cdots + \beta_k x_k$$

`statsmodels` finds the β's by **maximum likelihood**: the same idea as the likelihood function from Module 4, just with 16 parameters instead of one. Each β is the change in the log-odds of default when that feature goes up by 1 and the others stay the same. e^{β} is the **odds ratio**: above 1 means higher risk, below 1 means lower risk.

3a. Two models

- **Model A (profile only):** credit limit, age, sex, education, marital status. This is roughly what a bank knows when someone first applies.
- **Model B (profile + payment behavior):** Model A plus September status, the count of earlier late months, utilization and the two payment amounts.

In [5]:
```
profile_cols = ["LIMIT_10K", "AGE_10Y", "FEMALE", "EDU_UNIV", "EDU_HS",
"EDU_OTHER", "SINGLE", "MAR_OTHER"]

model_a = sm.Logit(y_train,
sm.add_constant(X_train[profile_cols])).fit(disp=0)
model_b = sm.Logit(y_train, sm.add_constant(X_train)).fit(disp=0)

print("Model A (profile only)")
print("  log-likelihood =", round(model_a.llf, 2), " AIC =",
round(model_a.aic, 2), " pseudo R^2 =", round(model_a.prsquared, 2))
print("Model B (profile + behavior)")
print("  log-likelihood =", round(model_b.llf, 2), " AIC =",
round(model_b.aic, 2), " pseudo R^2 =", round(model_b.prsquared, 2))
```

```
Model A (profile only)
  log-likelihood = -10726.67  AIC = 21471.34  pseudo R^2 = 0.03
Model B (profile + behavior)
  log-likelihood = -9050.58  AIC = 18133.15  pseudo R^2 = 0.18
```

3b. Likelihood ratio test

- H_0: the seven payment-behavior coefficients are all 0 (Model A is enough)
- H_a: at least one of them is not 0

The test statistic is LR = 2(ℓ_B – ℓ_A). If H_0 is true it follows a chi-squared distribution with df = number of extra coefficients = 7.

In [6]:
```
from scipy import stats

lr_stat = 2 * (model_b.llf - model_a.llf)
lr_df = int(model_b.df_model - model_a.df_model)
lr_p = stats.chi2.sf(lr_stat, lr_df)
print("LR statistic =", round(lr_stat, 2))
print("df =", lr_df)
print("P-value =", lr_p)
```

```
LR statistic = 3352.19
df = 7
P-value = 0.0
```

LR = **3,352.19** with df = 7, and the P-value is basically 0. So we **reject H_0**: payment behavior adds a lot of information on top of the client's profile. The other numbers agree:

- AIC drops from **21,471.34 to 18,133.15** (lower is better).
- Pseudo R^2 goes from **0.03 to 0.18**. The profile by itself explains very little.

We use **Model B** from here on.

3c. Model B coefficients and odds ratios

In [7]:
```
model_b.summary2().tables[1].round(3)
```

Out[7]:

	Coef.	Std.Err.	z	P>|z|	[0.025	0.975]
const	-0.964	0.122	-7.926	0.000	-1.203	-0.726
LIMIT_10K	-0.017	0.002	-8.978	0.000	-0.020	-0.013
AGE_10Y	0.051	0.024	2.148	0.032	0.004	0.097
FEMALE	-0.145	0.039	-3.744	0.000	-0.221	-0.069
EDU_UNIV	-0.007	0.045	-0.161	0.872	-0.095	0.080
EDU_HS	-0.048	0.060	-0.807	0.420	-0.165	0.069
EDU_OTHER	-1.293	0.242	-5.344	0.000	-1.767	-0.819
SINGLE	-0.127	0.044	-2.915	0.004	-0.213	-0.042
MAR_OTHER	-0.195	0.169	-1.152	0.249	-0.525	0.136
SEP_REVOLVING	-0.314	0.059	-5.343	0.000	-0.429	-0.199
SEP_LATE_1M	0.380	0.065	5.890	0.000	0.254	0.507
SEP_LATE_2PLUS	1.790	0.073	24.520	0.000	1.647	1.933
MONTHS_LATE_PRIOR	0.251	0.016	15.966	0.000	0.220	0.282
UTILIZATION	0.241	0.061	3.936	0.000	0.121	0.361
LOG_PAY_AMT1	-0.043	0.006	-6.690	0.000	-0.056	-0.031
LOG_PAY_AMT2	-0.039	0.006	-6.570	0.000	-0.051	-0.028

In [8]:
```
ci = model_b.conf_int()
odds = pd.DataFrame({
    "feature": [FEATURE_LABELS.get(c, c) for c in model_b.params.index],
    "coef": model_b.params.values,
    "odds_ratio": np.exp(model_b.params.values),
    "ci_low": np.exp(ci[0].values),
    "ci_high": np.exp(ci[1].values),
    "p_value": model_b.pvalues.values,
}, index=model_b.params.index).drop(index="const")
odds.round(3).to_csv(TABLE_DIR / "odds_ratios.csv")
odds.set_index("feature").round(3)
```

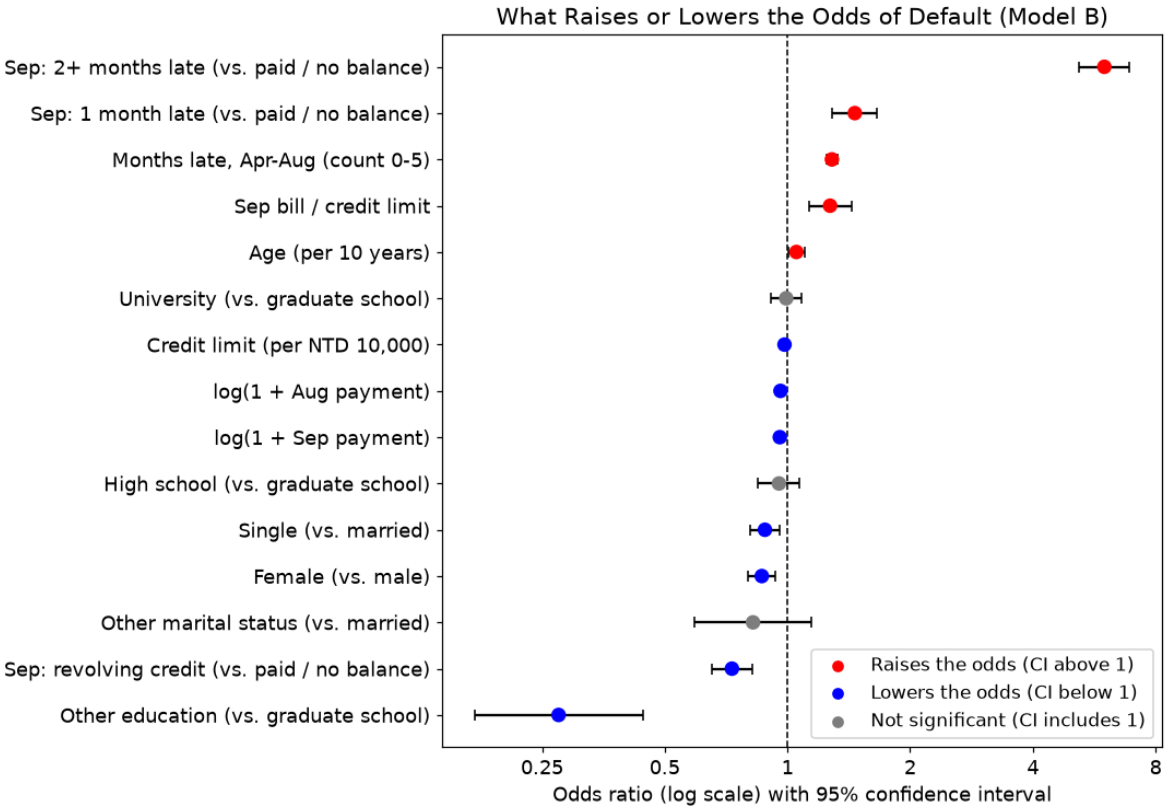
Out[8]:

feature	coef	odds_ratio	ci_low	ci_high	p_value
Credit limit (per NTD 10,000)	-0.017	0.983	0.980	0.987	0.000
Age (per 10 years)	0.051	1.052	1.004	1.102	0.032
Female (vs. male)	-0.145	0.865	0.802	0.933	0.000
University (vs. graduate school)	-0.007	0.993	0.910	1.084	0.872
High school (vs. graduate school)	-0.048	0.953	0.848	1.071	0.420
Other education (vs. graduate school)	-1.293	0.275	0.171	0.441	0.000
Single (vs. married)	-0.127	0.880	0.808	0.959	0.004
Other marital status (vs. married)	-0.195	0.823	0.591	1.146	0.249
Sep: revolving credit (vs. paid / no balance)	-0.314	0.731	0.651	0.820	0.000
Sep: 1 month late (vs. paid / no balance)	0.380	1.463	1.289	1.660	0.000
Sep: 2+ months late (vs. paid / no balance)	1.790	5.989	5.191	6.911	0.000
Months late, Apr-Aug (count 0-5)	0.251	1.285	1.246	1.326	0.000
Sep bill / credit limit	0.241	1.272	1.128	1.434	0.000
log(1 + Sep payment)	-0.043	0.958	0.946	0.970	0.000
log(1 + Aug payment)	-0.039	0.961	0.950	0.973	0.000

In [9]:

```
plot_df = odds.sort_values("odds_ratio")
colors = []
for low, high in zip(plot_df["ci_low"], plot_df["ci_high"]):
    if low > 1:
        colors.append("red")
    elif high < 1:
        colors.append("blue")
    else:
        colors.append("gray")

y_pos = np.arange(len(plot_df))
plt.figure(figsize=(10, 7))
plt.errorbar(plot_df["odds_ratio"], y_pos,
             xerr=[plot_df["odds_ratio"] - plot_df["ci_low"],
plot_df["ci_high"] - plot_df["odds_ratio"]],
             fmt="none", ecolor="black", capsize=3)
plt.scatter(plot_df["odds_ratio"], y_pos, color=colors, s=60, zorder=3)
plt.axvline(1, color="black", linestyle="--", linewidth=1)
plt.xscale("log")
plt.xticks([0.25, 0.5, 1, 2, 4, 8], ["0.25", "0.5", "1", "2", "4", "8"])
plt.minorticks_off()
plt.yticks(y_pos, plot_df["feature"])
plt.scatter([], [], color="red", label="Raises the odds (CI above 1)")
plt.scatter([], [], color="blue", label="Lowers the odds (CI below 1)")
plt.scatter([], [], color="gray", label="Not significant (CI includes
1)")
plt.legend(loc="lower right")
plt.xlabel("Odds ratio (log scale) with 95% confidence interval")
plt.title("What Raises or Lowers the Odds of Default (Model B)")
plt.tight_layout()
plt.savefig(FIG_DIR / "odds_ratios.png", dpi=150)
plt.show()
```


What Raises or Lowers the Odds of Default (Model B)
Sep: 2+ months late (vs. paid / no balance)
Sep: 1 month late (vs. paid / no balance)
Months late, Apr-Aug (count 0-5)
Sep bill / credit limit
Age (per 10 years)
University (vs. graduate school)
Credit limit (per NTD 10,000)
log(1 + Aug payment)
log(1 + Sep payment)
High school (vs. graduate school)
Single (vs. married)
Female (vs. male)
Other marital status (vs. married)
Sep: revolving credit (vs. paid / no balance)
Other education (vs. graduate school)
Raises the odds (CI above 1)
Lowers the odds (CI below 1)
Not significant (CI includes 1)
0.25
0.5
1
2
4
8
Odds ratio (log scale) with 95% confidence interval

How to read this: each dot is an odds ratio and the bar is its 95% CI. If the bar crosses the dashed line at 1, we can't say that feature changes the odds.

What **raises** the odds of default (keeping everything else the same):

- **2+ months late in September** multiplies the odds by **5.99** (95% CI 5.19 to 6.91) compared with paying in full or having no balance. **1 month late** multiplies them by **1.46**.
- **Each earlier late month** (April–August) multiplies the odds by **1.29**. A client who was late in all five earlier months has about $1.29^5 \approx$ **3.50 times** the odds of a client who was never late.
- **Using more of the limit:** going from a 0% to a 100% balance-to-limit ratio multiplies the odds by **1.27**.

What **lowers** the odds:

- **Credit limit:** each extra NTD 10,000 lowers the odds by about 1.7% (OR 0.983). That sounds small, but NTD 100,000 more means $0.983^{10} \approx$ **0.84**, about 16% lower odds. The bank probably gives bigger limits to clients it already trusts, so some of this is the bank's own screening.
- **Paying more:** paying 10 times more in September lowers the odds by about 9% (0.958 raised to ln 10 ≈ 2.30 is 0.91). August's payment has almost the same effect.
- **Revolving credit** (paid the minimum, not late) has lower odds than "paid / no balance" (OR **0.73**). This surprised us. We think it's because code -2 ("no balance") includes cards nobody is using, which may behave differently from cards in normal use.

Demographics:

- **Women** have about 13% lower odds than men (OR **0.87**), and **single** clients about 12% lower odds than married clients (OR **0.88**), after accounting for payment behavior.
- **Age** is just barely significant (OR 1.05 per 10 years, P = 0.03, CI 1.004 to 1.10).
- **University and high school** are not different from graduate school once payment behavior is in the model (P = 0.87 and 0.42).
- **"Other" education** has very low odds (OR 0.28). This small group (about 1.6% of clients) includes the undocumented codes 0, 5 and 6 that we merged, so we don't read much into it.

Takeaway: the three biggest effects are all about recent payment history, not about who the client is.

3d. Do we need sex, marital status and age?

Many lenders aren't allowed to use sex or marital status (and sometimes age) when they make credit decisions. So we refit Model B without them and compare. (We only use the test set here to report how much things change; it doesn't affect which model we pick.)

In [10]:
```
from sklearn.metrics import roc_auc_score

no_demo_cols = [c for c in X_train.columns if c not in ["FEMALE",
"SINGLE", "MAR_OTHER", "AGE_10Y"]]
model_c = sm.Logit(y_train,
sm.add_constant(X_train[no_demo_cols])).fit(disp=0)

auc_b = roc_auc_score(y_test, model_b.predict(sm.add_constant(X_test)))
auc_c = roc_auc_score(y_test,
model_c.predict(sm.add_constant(X_test[no_demo_cols])))
print("Model B (all 15 features):        test AUC =", round(auc_b, 4), "
AIC =", round(model_b.aic, 2))
print("Model C (no sex / marital / age): test AUC =", round(auc_c, 4), "
AIC =", round(model_c.aic, 2))
```

```
Model B (all 15 features):        test AUC = 0.7637  AIC = 18133.15
Model C (no sex / marital / age): test AUC = 0.7636  AIC = 18163.53
```

Taking out sex, marital status and age changes the test AUC from **0.7637 to 0.7636**, basically nothing. AIC goes up a little (18,133.15 → 18,163.53), so these columns carry a bit of information, but it doesn't help rank clients at all. A lender could leave them out and lose nothing, which is better for fairness and avoids legal problems.

4. Comparison models

We compare our logistic regression with:

- **Logistic regression on all 23 raw columns**, to check that our feature choices actually helped.
- **Decision tree** (max depth 5): one set of if/then rules.
- **Random forest:** 300 trees, each built on a random sample of the data, with their votes averaged.
- **Gradient boosting:** trees built one after another, each one fixing the mistakes of the ones before (the same idea as XGBoost from Module 5).

The tree models get the 23 raw columns. Trees can find their own cut points, so they don't need dummies or logs.

We score every model with **5-fold cross-validation ROC-AUC** on the training set. The training set is cut into 5 parts; each part takes a turn as a mini test set while the model trains on the other 4. **ROC-AUC** is the chance that the model gives a random defaulter a higher score than a random non-defaulter (0.5 = coin flip, 1 = perfect). It doesn't depend on a cutoff, which makes it a fair way to compare models when only 22% of clients default. We show AUC with 3 decimals because the differences between models are small.

4a. Picking the random forest's leaf size

`min_samples_leaf` is the smallest group of clients a tree is allowed to end on. Too small and the trees memorize the training data; too big and they miss patterns. We try a few values.

In [11]:
```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import StratifiedKFold, cross_val_score

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=500)
raw_cols = [c for c in df.columns if c != "DEFAULT"]
R_train = train[raw_cols]
R_test = test[raw_cols]

leaf_scores = {}
for leaf in [5, 10, 25, 50, 100]:
    rf = RandomForestClassifier(n_estimators=300, min_samples_leaf=leaf,
random_state=500, n_jobs=-1)
    scores = cross_val_score(rf, R_train, y_train, cv=cv,
scoring="roc_auc")
    leaf_scores[leaf] = scores.mean()
    print("min_samples_leaf =", leaf, " CV AUC =", round(scores.mean(),
4))

best_leaf = max(leaf_scores, key=leaf_scores.get)
print("Best min_samples_leaf =", best_leaf)
```

```
min_samples_leaf = 5  CV AUC = 0.7804
min_samples_leaf = 10  CV AUC = 0.7843
min_samples_leaf = 25  CV AUC = 0.7869
min_samples_leaf = 50  CV AUC = 0.7869
min_samples_leaf = 100  CV AUC = 0.7868
Best min_samples_leaf = 50
```

Leaf sizes from 25 to 100 all give about the same CV AUC (**0.787**). Smaller leaves do worse (0.780 at 5) because the trees start memorizing the training clients. The code picks the highest one, **min_samples_leaf = 50**, and we use it below.

4b. Cross-validation for all models

In [12]:
```
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier

# name: (model, training features, test features)
models = {
    "Logistic regression (our features)":
(make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)),
X_train, X_test),
    "Logistic regression (raw columns)":
(make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)),
R_train, R_test),
    "Decision tree": (DecisionTreeClassifier(max_depth=5,
min_samples_leaf=50, random_state=500), R_train, R_test),
    "Random forest": (RandomForestClassifier(n_estimators=300,
min_samples_leaf=best_leaf, random_state=500, n_jobs=-1), R_train,
R_test),
    "Gradient boosting":
(HistGradientBoostingClassifier(learning_rate=0.05, max_iter=300,
random_state=500), R_train, R_test),
}

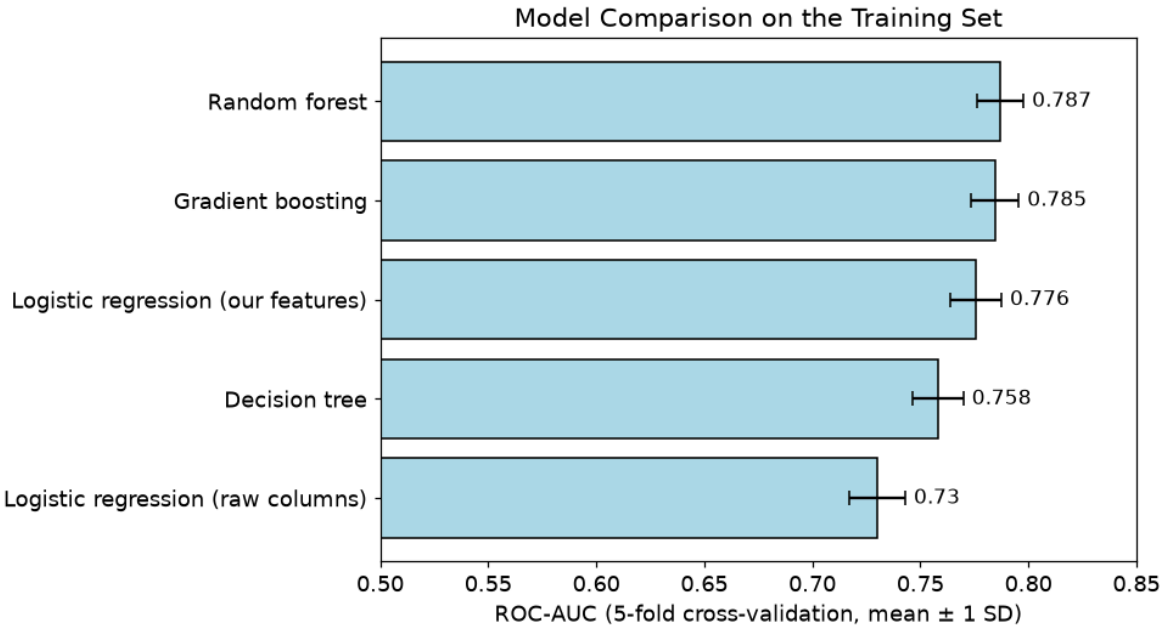
cv_mean = {}
cv_sd = {}
for name, (model, Xtr, Xte) in models.items():
    scores = cross_val_score(model, Xtr, y_train, cv=cv,
scoring="roc_auc")
    cv_mean[name] = scores.mean()
    cv_sd[name] = scores.std()
    print(name)
    print("  CV AUC =", round(scores.mean(), 3), " SD across folds =",
round(scores.std(), 3))

cv_results = pd.DataFrame({"cv_auc_mean": cv_mean, "cv_auc_sd":
cv_sd}).sort_values("cv_auc_mean")
cv_results.round(3).to_csv(TABLE_DIR / "cv_results.csv")
```

```
Logistic regression (our features)
  CV AUC = 0.776  SD across folds = 0.012
Logistic regression (raw columns)
  CV AUC = 0.73  SD across folds = 0.013
Decision tree
  CV AUC = 0.758  SD across folds = 0.012
Random forest
  CV AUC = 0.787  SD across folds = 0.011
```

```
Gradient boosting
  CV AUC = 0.785  SD across folds = 0.011
```

In [13]:
```
plt.figure(figsize=(9, 5))
plt.barh(cv_results.index, cv_results["cv_auc_mean"],
xerr=cv_results["cv_auc_sd"],
         color="lightblue", edgecolor="black", capsize=4)
for i, (m, sd) in enumerate(cv_results.values):
    plt.text(m + sd + 0.004, i, str(round(m, 3)), va="center",
fontsize=11)
plt.xlim(0.5, 0.85)
plt.xlabel("ROC-AUC (5-fold cross-validation, mean ± 1 SD)")
plt.title("Model Comparison on the Training Set")
plt.tight_layout()
plt.savefig(FIG_DIR / "cv_model_comparison.png", dpi=150)
plt.show()
```



- The **random forest** scored highest (**0.787**), with **gradient boosting** right behind (**0.785**).
- Our **logistic regression scored 0.776**, about **0.011 lower** than the random forest. The SD across folds is about 0.01, so the gap is about one SD: real, but small.
- The **logistic regression on raw columns** scored only **0.730**. Same method, but without our feature work it lost about 0.046. So grouping the repayment codes and fixing the skewed amounts helped **more** than switching to a more complex model.
- One **decision tree** (0.758) did worse than both tree ensembles, which is what we expected.

We take the **logistic regression (our features)** and the **random forest** into notebook 04.

5. Save predictions for notebook 04

- **Out-of-fold predictions on the training set:** every training client gets a probability from a model that didn't see them during training. Notebook 04 uses these to choose a cutoff, so the test set stays untouched until the very end.
- **Test-set predictions:** each model is trained on the full training set and then scores the test set.

In [14]:
```
from sklearn.model_selection import cross_val_predict

oof = pd.DataFrame({"y": y_train.values})
for name in ["Logistic regression (our features)", "Random forest"]:
    model, Xtr, Xte = models[name]
    oof[name] = cross_val_predict(model, Xtr, y_train, cv=cv,
method="predict_proba")[:, 1]
oof.to_csv(PROCESSED_DIR / "train_oof_predictions.csv", index=False)

test_pred = pd.DataFrame({"y": y_test.values})
for name, (model, Xtr, Xte) in models.items():
    model.fit(Xtr, y_train)
    test_pred[name] = model.predict_proba(Xte)[:, 1]
test_pred.to_csv(PROCESSED_DIR / "test_predictions.csv", index=False)

print("Saved", len(oof), "out-of-fold predictions and", len(test_pred),
"test predictions")
```

```
Saved 20975 out-of-fold predictions and 8990 test predictions
```

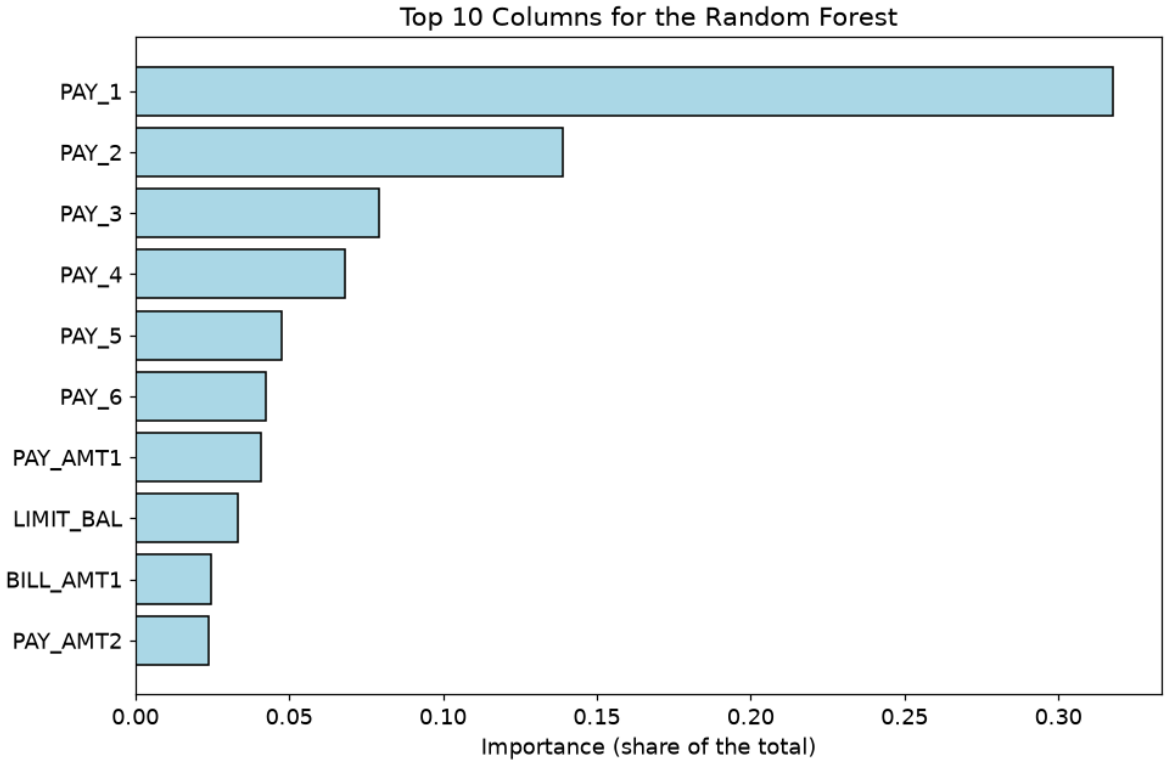
6. What does the random forest pay attention to?

A random forest doesn't have odds ratios. Instead, it can tell us how much each column helped its trees split the clients into "default" and "no default" groups (the importance values add up to 1).

In [15]:
```
rf_model = models["Random forest"][0]
importance = pd.Series(rf_model.feature_importances_,
index=raw_cols).sort_values()
top = importance.tail(10)

plt.figure(figsize=(9, 6))
plt.barh(top.index, top.values, color="lightblue", edgecolor="black")
plt.xlabel("Importance (share of the total)")
plt.title("Top 10 Columns for the Random Forest")
plt.tight_layout()
plt.savefig(FIG_DIR / "rf_feature_importance.png", dpi=150)
plt.show()

print(importance.sort_values(ascending=False).head(10).round(3).to_strin
g())
```




```
PAY_1        0.318
PAY_2        0.139
PAY_3        0.079
PAY_4        0.068
PAY_5        0.048
PAY_6        0.042
PAY_AMT1     0.041
LIMIT_BAL    0.033
BILL_AMT1    0.025
PAY_AMT2     0.024
```

The random forest pays attention to the same things as the logistic regression. **September repayment status (PAY_1)** is by far the most important column (**0.32**), followed by the other five months of repayment status, then September's payment, the credit limit and the bill. Sex, education, marital status and age don't make the top 10. Two very different models agreeing on what matters makes us more confident in this finding.

04 — Evaluation, Choosing a Cutoff, and What It Means

AAI-500 Final Team Project, Group 02 — Predicting Credit Card Default

Names: Abhineet Sood, Ravi Kanth Vuddagiri Date: October 4, 2026

Notebook 03 picked two finalists using cross-validation on the training set:

- **Logistic regression (our features):** the model we can explain
- **Random forest:** the model with the best score

Here we score them on the **test set** (8,990 clients the models have never seen), then choose a cutoff (threshold) based on what each kind of mistake costs a lender.

In [1]:
```
import sys
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

PROJECT_ROOT = Path.cwd().resolve().parent if Path.cwd().name ==
"notebooks" else Path.cwd().resolve()
sys.path.insert(0, str(PROJECT_ROOT))

from src.data_loader import PROCESSED_DIR

plt.rcParams.update({"figure.dpi": 110, "font.size": 12,
"axes.titlesize": 14,
                     "axes.labelsize": 12, "legend.fontsize": 11})
FIG_DIR = PROJECT_ROOT / "reports" / "figures"
TABLE_DIR = PROJECT_ROOT / "reports" / "tables"

LOGIT = "Logistic regression (our features)"
RF = "Random forest"
```

1. Load the predictions

Both files come from notebook 03. Each column is one model's predicted probability of default.

In [2]:
```
test_pred = pd.read_csv(PROCESSED_DIR / "test_predictions.csv")
oof = pd.read_csv(PROCESSED_DIR / "train_oof_predictions.csv")
y_test = test_pred["y"]
model_names = [c for c in test_pred.columns if c != "y"]

print("Test clients =", len(test_pred))
print("Defaults in the test set =", y_test.sum(), " rate =",
round(y_test.mean(), 4))
```

```
Test clients = 8990
Defaults in the test set = 1989  rate = 0.2212
```

2. Test-set results at the usual 0.5 cutoff

If a model's probability is 0.5 or more, we predict "default".

- **Accuracy:** share of all clients predicted correctly. This is misleading here, since always saying "no default" already gets about 78%.
- **Precision:** of the clients we flag, how many actually default.
- **Recall:** of the clients who actually default, how many we flag.
- **F1:** one number that balances precision and recall.
- **ROC-AUC:** how well the model ranks defaulters above non-defaulters (doesn't need a cutoff).

In [3]:

```
from sklearn.metrics import accuracy_score, f1_score, precision_score,
recall_score, roc_auc_score

def results_at(y, prob, cutoff):
    pred = (prob >= cutoff).astype(int)
    return {"roc_auc": roc_auc_score(y, prob),
            "accuracy": accuracy_score(y, pred),
            "precision": precision_score(y, pred),
            "recall": recall_score(y, pred),
            "f1": f1_score(y, pred),
            "share_flagged": pred.mean()}

rows = {}
for name in model_names:
    rows[name] = results_at(y_test, test_pred[name], 0.5)
test_results = pd.DataFrame(rows).T.sort_values("roc_auc",
ascending=False)
test_results.round(3).to_csv(TABLE_DIR / "test_results_cutoff_0.5.csv")
test_results.round(3)
```

Out[3]:

	roc_auc	accuracy	precision	recall	f1	share_flagged
Gradient boosting	0.773	0.816	0.658	0.355	0.461	0.119
Random forest	0.773	0.814	0.652	0.346	0.452	0.118
Logistic regression (our features)	0.764	0.814	0.654	0.338	0.445	0.114
Decision tree	0.750	0.815	0.664	0.332	0.443	0.111
Logistic regression (raw columns)	0.708	0.807	0.679	0.242	0.357	0.079

- On ROC-AUC, **random forest and gradient boosting tie at 0.773**, and our **logistic regression gets 0.764**. These are close to the cross-validation scores from notebook 03 (0.787 and 0.776), so the models weren't overfit to the training data.
- At the 0.5 cutoff, every model has low **recall: only 33–36% of defaulters get flagged**. Accuracy is about 81%, only 3 points better than always guessing "no default".
- That's not because the models are bad at ranking clients. It's because 0.5 is the wrong cutoff when only 22% of clients default and missing a defaulter is expensive. Section 5 fixes this.

3. ROC curves

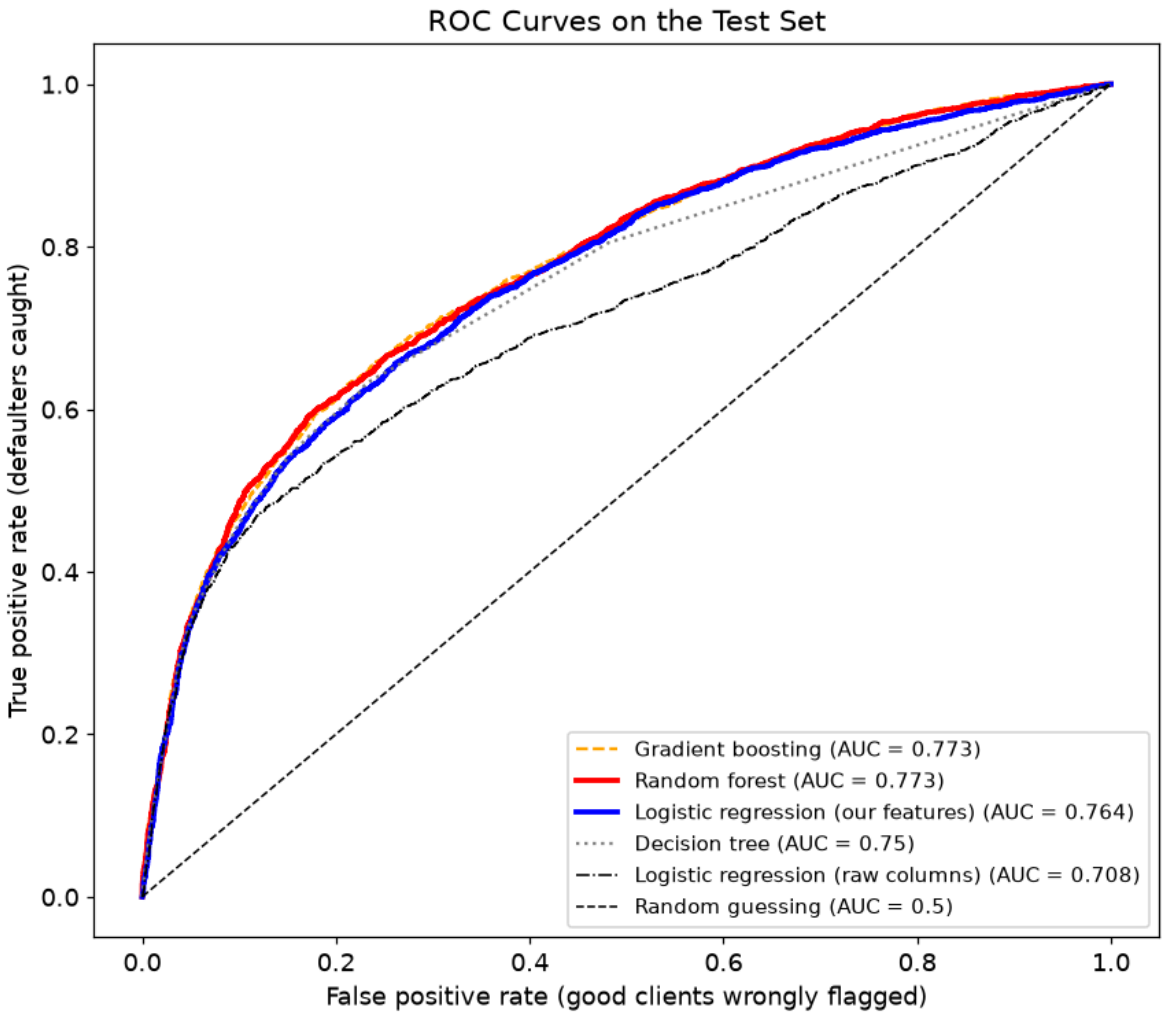
An ROC curve shows every possible cutoff at once: how many defaulters we catch (up) vs. how many good clients we wrongly flag (right). The closer the curve is to the top-left corner, the better.

In [4]:

```
from sklearn.metrics import roc_curve

styles = {LOGIT: ("blue", "-", 2.5), RF: ("red", "-", 2.5), "Gradient
boosting": ("orange", "--", 1.5),
          "Decision tree": ("gray", ":", 1.5), "Logistic regression (raw
columns)": ("black", "-.", 1)}

plt.figure(figsize=(8, 7))
for name in test_results.index:
    fpr, tpr, cutoffs = roc_curve(y_test, test_pred[name])
    color, line, width = styles[name]
    plt.plot(fpr, tpr, color=color, linestyle=line, linewidth=width,
             label=name + " (AUC = " + str(round(test_results.loc[name,
"roc_auc"], 3)) + ")")
plt.plot([0, 1], [0, 1], color="black", linestyle="--", linewidth=1,
label="Random guessing (AUC = 0.5)")
plt.xlabel("False positive rate (good clients wrongly flagged)")
plt.ylabel("True positive rate (defaulters caught)")
plt.title("ROC Curves on the Test Set")
plt.legend(loc="lower right", fontsize=10)
plt.tight_layout()
plt.savefig(FIG_DIR / "roc_curves.png", dpi=150)
plt.show()
```



The curves for the random forest, gradient boosting and our logistic regression are almost on top of each other. All of them are well above the diagonal line, which is what random guessing would give.

4. Is the random forest really better? Bootstrap confidence intervals

The test set is just one sample of clients, so its AUC has sampling error. We use the bootstrap: resample the test set with replacement 2,000 times, recompute each AUC, and take the middle 95% of the results as a confidence interval. We use the **same** resampled clients for both models, so we can also get a CI for the **difference**.

In [5]:
```
rng = np.random.default_rng(500)
n = len(y_test)
y_arr = y_test.to_numpy()
p_logit = test_pred[LOGIT].to_numpy()
p_rf = test_pred[RF].to_numpy()

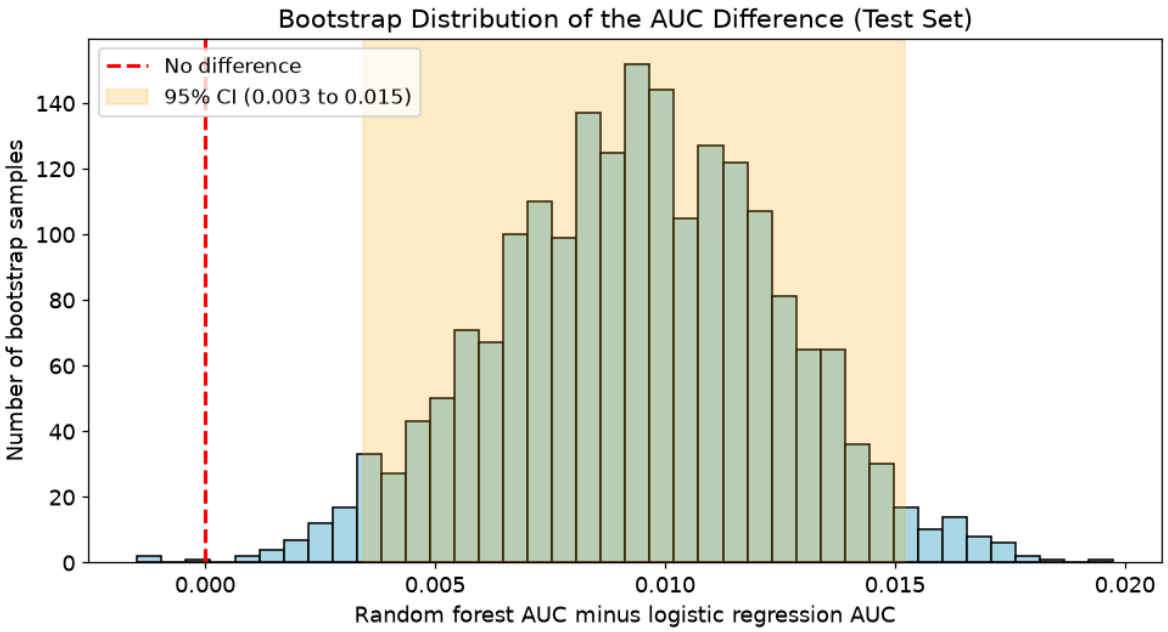
boot_logit = []
boot_rf = []
boot_diff = []
for i in range(2000):
    idx = rng.integers(0, n, n)
    auc_logit = roc_auc_score(y_arr[idx], p_logit[idx])
    auc_rf = roc_auc_score(y_arr[idx], p_rf[idx])
    boot_logit.append(auc_logit)
    boot_rf.append(auc_rf)
    boot_diff.append(auc_rf - auc_logit)

print("Logistic regression AUC: 95% CI =",
np.round(np.percentile(boot_logit, [2.5, 97.5]), 3))
print("Random forest AUC:       95% CI =",
np.round(np.percentile(boot_rf, [2.5, 97.5]), 3))
print("Difference (RF - logistic): 95% CI =",
np.round(np.percentile(boot_diff, [2.5, 97.5]), 3))
print("Share of resamples where RF is higher =",
round(np.mean(np.array(boot_diff) > 0), 3))
```

```
Logistic regression AUC: 95% CI = [0.751 0.776]
Random forest AUC:       95% CI = [0.761 0.785]
Difference (RF - logistic): 95% CI = [0.003 0.015]
Share of resamples where RF is higher = 0.998
```

In [6]:
```
low, high = np.percentile(boot_diff, [2.5, 97.5])

plt.figure(figsize=(9, 5))
plt.hist(boot_diff, bins=40, color="lightblue", edgecolor="black")
plt.axvline(0, color="red", linestyle="--", linewidth=2, label="No
difference")
plt.axvspan(low, high, color="orange", alpha=0.2,
            label="95% CI (" + str(round(low, 3)) + " to " +
str(round(high, 3)) + ")")
plt.xlabel("Random forest AUC minus logistic regression AUC")
plt.ylabel("Number of bootstrap samples")
plt.title("Bootstrap Distribution of the AUC Difference (Test Set)")
plt.legend(loc="upper left")
plt.tight_layout()
plt.savefig(FIG_DIR / "bootstrap_auc_difference.png", dpi=150)
plt.show()
```



- Logistic regression test AUC: **0.764**, 95% CI **0.751 to 0.776**
- Random forest test AUC: **0.773**, 95% CI **0.761 to 0.785**
- Difference: 95% CI **0.003 to 0.015**. The interval doesn't include 0, and the random forest is higher in 99.8% of the resamples.

So the random forest really is better, but only by about **0.01 AUC**. The two separate intervals overlap a lot. We can only see the difference clearly because both models were scored on the same clients each time (a paired comparison).

5. Choosing a cutoff based on cost

A 0.5 cutoff treats both kinds of mistakes as equally bad. For a lender they aren't:

- **Missed defaulter (false negative):** the client stops paying and the bank loses the balance. Among clients who defaulted, the median September bill was about NTD 20,000.
- **False alarm (false positive):** a good client gets a call, a reminder or a lower limit. This costs some staff time and goodwill, but much less than a lost balance.

We **assume a missed defaulter costs 5 times as much as a false alarm**. We don't have the bank's real numbers, so section 6 checks how the answer changes with other ratios.

Expected cost per client at cutoff t:

cost(t) = (5 × number of missed defaulters + 1 × number of false alarms) / number of clients

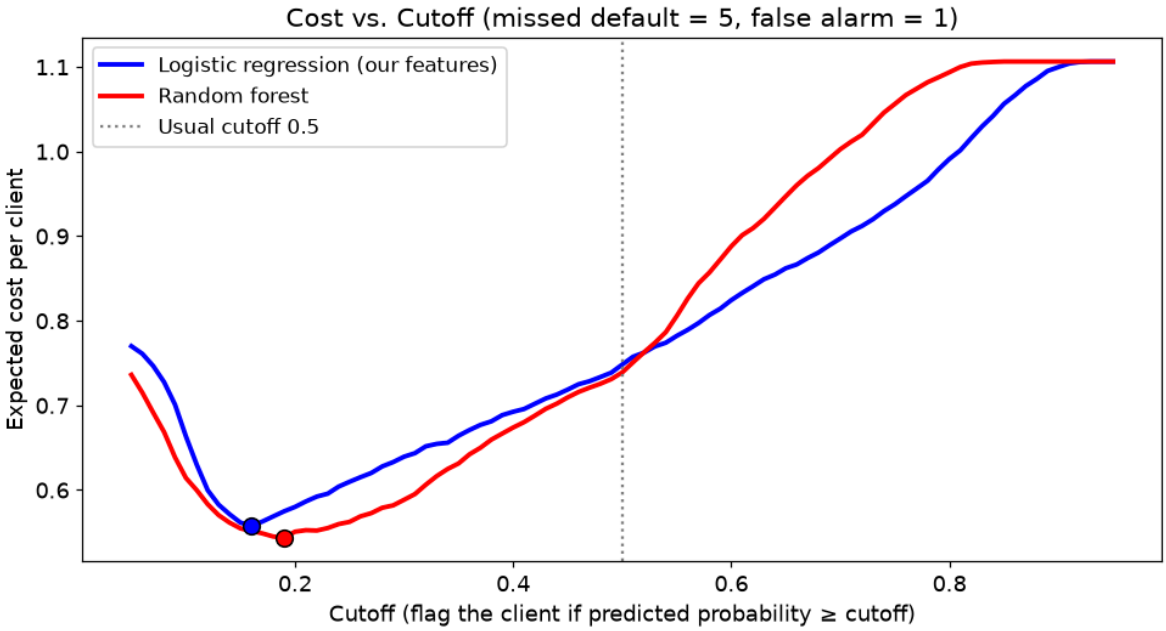
We pick t using the **out-of-fold training predictions** from notebook 03, then check it on the test set. That way the test set never affects the choice.

In [7]:
```
COST_MISS = 5
COST_FALSE_ALARM = 1
cutoffs = np.round(np.arange(0.05, 0.96, 0.01), 2)

def cost_at_each_cutoff(y, prob, cost_miss, cost_false_alarm):
    costs = []
    for t in cutoffs:
        pred = (prob >= t).astype(int)
        missed = ((pred == 0) & (y == 1)).sum()
        false_alarms = ((pred == 1) & (y == 0)).sum()
        costs.append((cost_miss * missed + cost_false_alarm *
false_alarms) / len(y))
    return np.array(costs)

best_cutoff = {}
plt.figure(figsize=(9, 5))
for name, color in [(LOGIT, "blue"), (RF, "red")]:
    costs = cost_at_each_cutoff(oof["y"].to_numpy(),
oof[name].to_numpy(), COST_MISS, COST_FALSE_ALARM)
    best_cutoff[name] = cutoffs[costs.argmin()]
    plt.plot(cutoffs, costs, color=color, linewidth=2.5, label=name)
    plt.scatter(best_cutoff[name], costs.min(), color=color, s=80,
zorder=3, edgecolor="black")
    print(name, " best cutoff =", best_cutoff[name], " cost per client
=", round(costs.min(), 3))
plt.axvline(0.5, color="gray", linestyle=":", label="Usual cutoff 0.5")
plt.xlabel("Cutoff (flag the client if predicted probability ≥ cutoff)")
plt.ylabel("Expected cost per client")
plt.title("Cost vs. Cutoff (missed default = 5, false alarm = 1)")
plt.legend(loc="upper left")
plt.tight_layout()
plt.savefig(FIG_DIR / "cost_vs_threshold.png", dpi=150)
plt.show()
```

```
Logistic regression (our features)  best cutoff = 0.16  cost per client =
0.558
Random forest  best cutoff = 0.19  cost per client = 0.543
```


Cost vs. Cutoff (missed default = 5, false alarm = 1)
Logistic regression (our features)
Random forest
Usual cutoff 0.5
Expected cost per client
1.1
1.0
0.9
0.8
0.7
0.6
0.2
0.4
0.6
0.8
Cutoff (flag the client if predicted probability ≥ cutoff)

The lowest expected cost is at a cutoff of **0.16** for the logistic regression and **0.19** for the random forest, much lower than 0.5.

This matches what theory says. If the probabilities are accurate, flagging a client is worth it when p × 5 > (1 – p) × 1, which works out to **p > 1/6 ≈ 0.17**. Our data-driven cutoff landed right next to it. The curve is also pretty flat between about 0.13 and 0.20, so the exact value isn't critical.

Test-set results at 0.5 vs. the cost-based cutoff

In [8]:
```
rows = {}
for name in [LOGIT, RF]:
    for t in [0.5, best_cutoff[name]]:
        r = results_at(y_test, test_pred[name], t)
        pred = (test_pred[name] >= t).astype(int)
        missed = ((pred == 0) & (y_test == 1)).sum()
        false_alarms = ((pred == 1) & (y_test == 0)).sum()
        r["cost_per_client"] = (COST_MISS * missed + COST_FALSE_ALARM *
false_alarms) / len(y_test)
        rows[name + " @ " + str(t)] = r
cutoff_compare = pd.DataFrame(rows).T[["precision", "recall", "f1",
"share_flagged", "cost_per_client"]]
cutoff_compare.round(3).to_csv(TABLE_DIR / "test_cutoff_comparison.csv")
cutoff_compare.round(3)
```

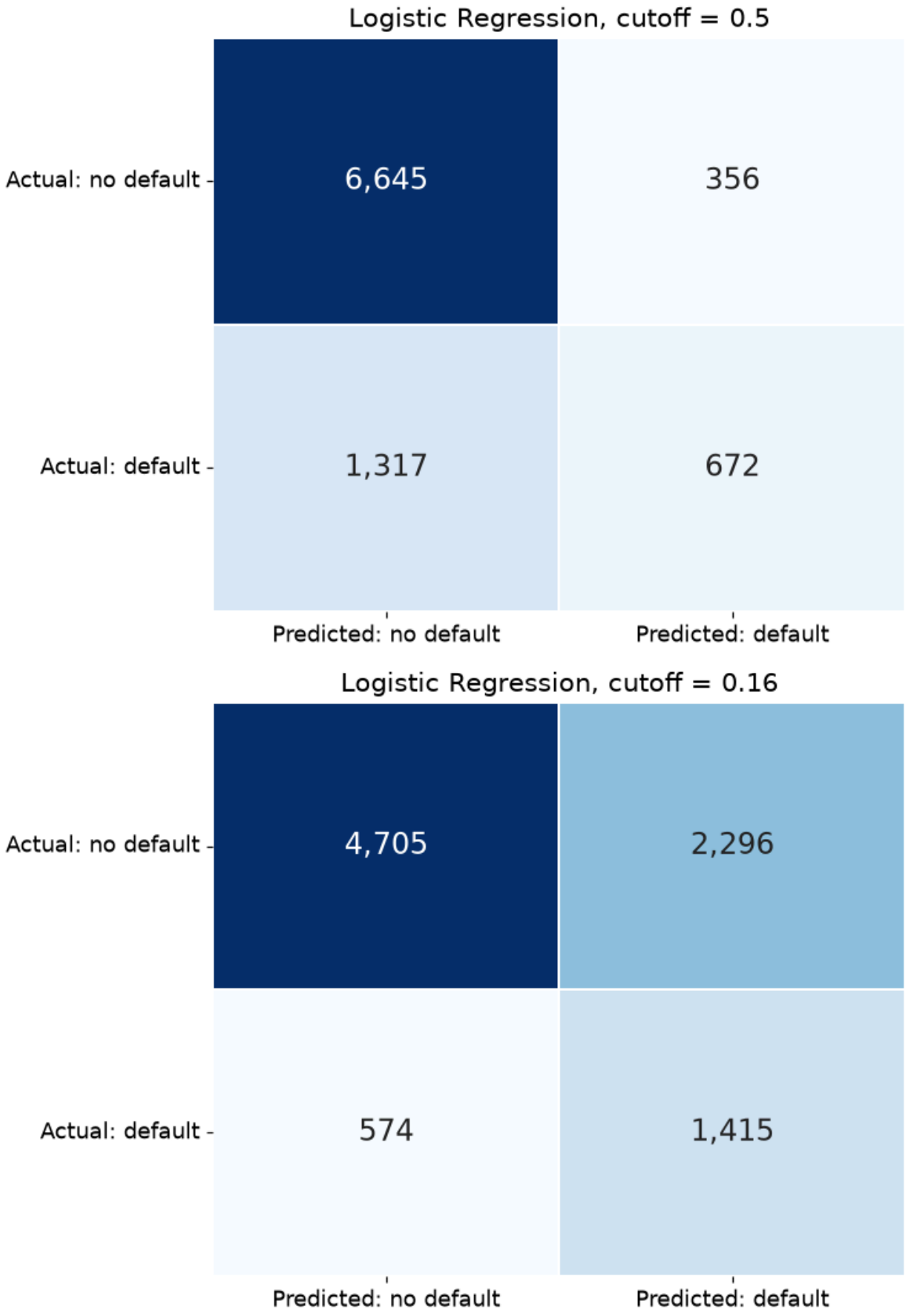
Out[8]:

	precision	recall	f1	share_flagged	cost_per_client
Logistic regression (our features) @ 0.5	0.654	0.338	0.445	0.114	0.772
Logistic regression (our features) @ 0.16	0.381	0.711	0.496	0.413	0.575
Random forest @ 0.5	0.652	0.346	0.452	0.118	0.764
Random forest @ 0.19	0.400	0.696	0.508	0.385	0.567

In [9]:

```
from sklearn.metrics import confusion_matrix
import seaborn as sns

fig, axes = plt.subplots(2, 1, figsize=(7, 10))
for ax, t in zip(axes, [0.5, best_cutoff[LOGIT]]):
    cm = confusion_matrix(y_test, (test_pred[LOGIT] >= t).astype(int))
    sns.heatmap(cm, annot=True, fmt=",", cmap="Blues", cbar=False,
ax=ax,
                annot_kws={"size": 16}, linewidths=1, linecolor="white")
    ax.set_xticklabels(["Predicted: no default", "Predicted: default"])
    ax.set_yticklabels(["Actual: no default", "Actual: default"],
rotation=0)
    ax.set_title("Logistic Regression, cutoff = " + str(t))
plt.tight_layout()
plt.savefig(FIG_DIR / "confusion_matrices_logit.png", dpi=150)
plt.show()
```


Logistic Regression, cutoff = 0.5
Actual: no default
Actual: default
6,645
356
1,317
672
Predicted: no default
Predicted: default
Logistic Regression, cutoff = 0.16
Actual: no default
Actual: default
4,705
2,296
574
1,415
Predicted: no default
Predicted: default

Moving the logistic regression's cutoff from 0.5 to 0.16 on the test set:

- **Recall goes from 34% to 71%**, so we catch about twice as many defaulters.
- **Precision drops from 65% to 38%**, so more of the flagged clients are false alarms.
- **The share of clients flagged goes from 11% to 41%.**
- **Expected cost per client drops from 0.77 to 0.58, about 25% lower.**

The random forest at its own cutoff (0.19) costs 0.567 per client vs. 0.575 for the logistic regression, about 1.4% lower.

6. How much does the cost assumption matter?

In [10]:
```
rows = []
for ratio in [2, 3, 5, 10]:
    costs = cost_at_each_cutoff(oof["y"].to_numpy(),
oof[LOGIT].to_numpy(), ratio, 1)
    t = cutoffs[costs.argmin()]
    r = results_at(y_test, test_pred[LOGIT], t)
    rows.append({"cost ratio (miss : false alarm)": str(ratio) + " : 1",
"best cutoff": t,
                 "theory 1/(1+ratio)": round(1 / (1 + ratio), 2),
"share_flagged": r["share_flagged"],
                 "recall": r["recall"], "precision": r["precision"]})
sensitivity = pd.DataFrame(rows).set_index("cost ratio (miss : false
alarm)")
sensitivity.round(3).to_csv(TABLE_DIR / "cutoff_sensitivity.csv")
sensitivity.round(2)
```

Out[10]:

cost ratio (miss : false alarm)	best cutoff	theory 1/(1+ratio)	share_flagged	recall	precision
2 : 1	0.34	0.33	0.16	0.43	0.58
3 : 1	0.23	0.25	0.25	0.55	0.49
5 : 1	0.16	0.17	0.41	0.71	0.38
10 : 1	0.11	0.09	0.76	0.92	0.27

The cost ratio matters a lot. If a missed default only costs twice as much as a false alarm, the best cutoff is **0.34**: we flag 16% of clients and catch 43% of defaulters. At 10 : 1 the cutoff drops to **0.11**, and we flag 76% of clients to catch 92% of defaulters. Each time, the cutoff is close to the theory value 1 / (1 + ratio).

The bank should plug in its own cost numbers. The model stays the same; only the cutoff changes.

7. Does the model sort clients well? Risk deciles

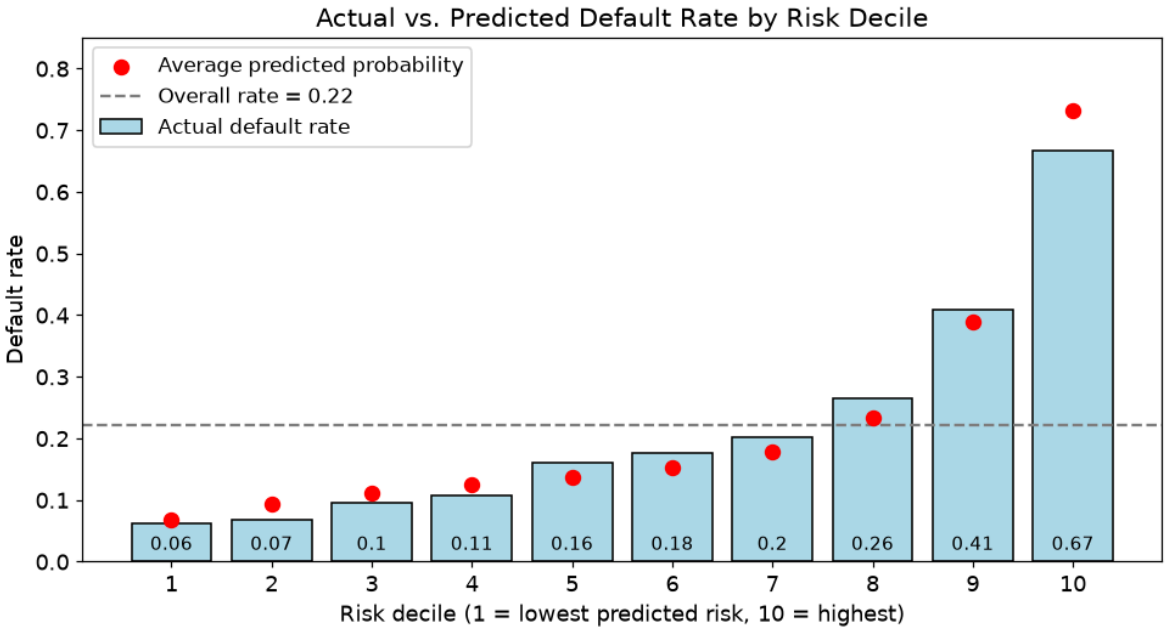
We sort the test clients into 10 equal groups by predicted probability, from lowest risk (1) to highest (10). For each group we compare the **actual** default rate (bars) with the model's **average predicted** probability (dots). If the dots sit on top of the bars, the probabilities can be trusted as they are.

In [11]:
```
deciles = pd.qcut(test_pred[LOGIT].rank(method="first"), 10,
labels=range(1, 11))
by_decile = test_pred.groupby(deciles, observed=True).agg(actual=("y",
"mean"), predicted=(LOGIT, "mean"),
                                                          defaulters=("y",
"sum"))
print(by_decile.round(2))
print()
print("Share of all defaulters in the top 2 deciles =",
round(by_decile["defaulters"].iloc[-2:].sum() / y_test.sum(), 2))
print("Share of all defaulters in the top 3 deciles =",
round(by_decile["defaulters"].iloc[-3:].sum() / y_test.sum(), 2))

x = np.arange(1, 11)
plt.figure(figsize=(9, 5))
plt.bar(x, by_decile["actual"], color="lightblue", edgecolor="black",
label="Actual default rate")
plt.scatter(x, by_decile["predicted"], color="red", s=60, zorder=3,
label="Average predicted probability")
for i, v in zip(x, by_decile["actual"]):
    plt.text(i, 0.012, str(round(v, 2)), ha="center", va="bottom",
fontsize=10)
plt.axhline(y_test.mean(), color="gray", linestyle="--", label="Overall
rate = " + str(round(y_test.mean(), 2)))
plt.xticks(x)
plt.ylim(0, 0.85)
plt.xlabel("Risk decile (1 = lowest predicted risk, 10 = highest)")
plt.ylabel("Default rate")
plt.title("Actual vs. Predicted Default Rate by Risk Decile")
plt.legend(loc="upper left")
plt.tight_layout()
plt.savefig(FIG_DIR / "default_rate_by_risk_decile.png", dpi=150)
plt.show()
```

```
                                    actual  predicted  defaulters
Logistic regression (our features)
1                                     0.06       0.07          55
2                                     0.07       0.09          61
3                                     0.10       0.11          86
4                                     0.11       0.12          97
5                                     0.16       0.14         145
6                                     0.18       0.15         158
7                                     0.20       0.18         181
8                                     0.26       0.23         238
9                                     0.41       0.39         368
10                                    0.67       0.73         600

Share of all defaulters in the top 2 deciles = 0.49
Share of all defaulters in the top 3 deciles = 0.61
```



The model sorts clients well. The **lowest-risk 10% default only 6%** of the time, while the **highest-risk 10% default 67%** of the time, about 3 times the overall rate. The **top two deciles (20% of clients) contain 49% of all defaulters**, and the top three contain 61%.

The red dots sit close to the tops of the bars, so the predicted probabilities are about right. The biggest gap is in the riskiest group, where the model predicts 0.73 and the actual rate is 0.67.

8. What this means for a lender

The logistic regression at the cost-based cutoff, scaled to 1,000 clients:

In [12]:

```
pred = (test_pred[LOGIT] >= best_cutoff[LOGIT]).astype(int)
tn, fp, fn, tp = confusion_matrix(y_test, pred).ravel()
scale = 1000 / len(y_test)

print("Out of 1,000 clients:")
print("  Actual defaulters =", round((tp + fn) * scale))
print("  Flagged by the model =", round((tp + fp) * scale))
print("  Defaulters caught =", round(tp * scale))
print("  Defaulters missed =", round(fn * scale))
print("  Good clients flagged (false alarms) =", round(fp * scale))
print("  Good clients left alone =", round(tn * scale))
```

```
Out of 1,000 clients:
  Actual defaulters = 221
  Flagged by the model = 413
  Defaulters caught = 157
  Defaulters missed = 64
  Good clients flagged (false alarms) = 255
  Good clients left alone = 523
```

Out of every 1,000 clients, about 221 will default. The model flags 413 of them. It catches **157** of the defaulters and misses 64. The price is about 255 false alarms, which is why the bank should use low-cost actions (a reminder, a call) for most flagged clients, and save stronger actions like lowering a limit for the top-risk decile.

9. Summary

	Logistic regression (our features)	Random forest
Test ROC-AUC (95% CI)	0.764 (0.751 to 0.776)	0.773 (0.761 to 0.785)
Cost-based cutoff (5 : 1)	0.16	0.19
Recall at that cutoff	71%	70%
Precision at that cutoff	38%	40%
Expected cost per client	0.575	0.567
Can it explain its decisions?	Yes: odds ratios, P-values, CIs	Only roughly (feature importance)

Why we recommend the logistic regression (the same question as in Module 5: why choose the explainable model when a black-box model is a bit more accurate?):

- The random forest is better by about **0.01 AUC** and **1.4% in cost**. That's real, but small.
- The logistic regression can say **why** a client is risky: "2+ months late in September multiplies the odds by about 6." A bank can check that, explain it to the client, and show it to regulators.
- Lenders often have to give a reason when they turn someone down or lower a limit. A random forest averages 300 trees, so there's no simple reason to give.
- We'd still keep the random forest as a benchmark, to notice if the simple model starts falling behind.

Main findings

1. **Recent repayment status is the strongest warning sign.** 2+ months late in September multiplies the odds of default by about 6, and each earlier late month by about 1.3.
2. **Credit limit and payment amounts matter, but less.** Higher limits and bigger payments both lower risk.
3. **Demographics add almost nothing.** Taking out sex, marital status and age changes the AUC by 0.0001.

4. **The cutoff matters as much as the model.** Moving from 0.5 to 0.16 doubles the share of defaulters caught and lowers expected cost by about 25%.